

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2023

Huấn luyện viên: Yang Yaoliang, Peng Sijin

China Computer Federation, 2023

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2023

IOI China candidate team papers / National training team 2023 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2023. Tập được tạo bằng LaTeX với lớp văn bản Unicode đọc được; mục lục và các trang thân bài đã kiểm tra đều trích xuất tốt bằng pdftotext. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục và ba bài đầu tiên, đồng thời ghi lại các chủ đề chính để phục vụ bản dịch chi tiết hơn.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2023*. Trang bìa ghi huấn luyện viên là Yang Yaoliang và Peng Sijin. Tập PDF gồm 249 trang và được tạo bằng LaTeX với gói hyperref.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Tổng quan một số phương pháp xử lý tính liên thông và các bài toán liên quan trong lý thuyết đồ thị	Wan Chengzhang
31	Bàn về xây dựng và sinh dữ liệu trong thi tin học	Liu Yiping
50	Bàn về một lớp bài toán có treewidth bị chặn	Wu Chang
63	Bao hàm–loại trừ trên quan hệ đôi một khác nhau	Zhou Ziheng
70	Báo cáo ra đề <i>Nhà ảo thuật</i>	Meng Yuhao
85	Hình học trong OI	Chang Ruinian
104	Bàn về các thuật toán liên quan tới xâu palindrome và ứng dụng	Xu Anyi
113	Bàn về một số ứng dụng của trường hữu hạn trong OI	Qi Langrui
123	Bàn về một lớp bài toán thống kê trên cây	Zhu Yikai
131	Bàn về ứng dụng của ma trận trong thi tin học	Yang Ningyuan
147	Bàn về một số bài toán liên quan tới matching đồ thị hai phía	Ke Yisi
156	Bàn về ứng dụng đặc biệt của DSU trong OI	Wang Xiangwen
167	Bàn về ứng dụng static top tree trên cây và đồ thị nối tiếp–song song tổng quát	Cheng Siyuan
181	Bước đầu về hypercube và các thuật toán liên quan	Guan Yanru
198	Bàn về một số phương pháp phân tích thừa số nguyên tố	Luo Siyuan
209	Một lớp cấu trúc dữ liệu xâu con cơ bản	Xu Tingqiang
220	Xem xét lại một vài bài toán số học cổ điển	Zheng Xuanye
232	Bàn về ứng dụng tính lồi của hàm số trong OI	Guo Yuchong

3 Tổng quan một số phương pháp xử lý tính liên thông và các bài toán liên quan trong lý thuyết đồ thị

Trường Trung học phổ thông số 2 trực thuộc Đại học Sư phạm Hoa Đông
Wan Chengzhang

Tóm tắt

Bài viết này chia thành hai phần: đồ thị vô hướng và đồ thị có hướng. Nội dung thảo luận một số tư tưởng thuật toán về tính liên thông có tính thực dụng trong thi tin học; ngoài ra còn khảo sát thêm một số bài toán lý thuyết đồ thị có liên hệ nhất định với tính liên thông.

3.1 Định nghĩa và quy ước

Với đồ thị vô hướng hoặc có hướng G :

- mặc định ký hiệu V là tập đỉnh của G , E là tập cạnh của G , $n = |V|$ là số đỉnh của G , $m = |E|$ là số cạnh của G ;
- nếu E không có phần tử lặp, ta nói G không có cạnh bội; nếu mọi $(u, u) \notin E$, ta nói G không có khuyên;
- G là đồ thị đơn khi và chỉ khi G không có cạnh bội và không có khuyên;
- ký hiệu $u \rightarrow v$ biểu thị một cạnh (u, v) ;
- ký hiệu $u \longrightarrow v$ biểu thị một đường đi bắt đầu tại u và kết thúc tại v , tức $u = x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_k = v$;
- một chu trình là một đường đi dạng $u \rightarrow u$;
- đường đi đơn là đường đi không đi qua đỉnh lặp;
- chu trình đơn là chu trình không đi qua đỉnh lặp, ngoại trừ đỉnh đầu và đỉnh cuối;
- $G' = (V', E')$ là đồ thị con của G khi và chỉ khi $V' \subseteq V$, $E' \subseteq E$;
- đồ thị con cảm sinh theo tập đỉnh V' của G là $G' = (V', E')$, $E' = \{(u, v) \in E \mid u, v \in V'\}$;
- đồ thị con cảm sinh theo tập cạnh E' của G là $G' = (V', E')$, $V' = \{x \in V \mid (\exists y)((x, y) \in E' \vee (y, x) \in E')\}$.

3.2 Đồ thị vô hướng

Tính liên thông trong đồ thị vô hướng là một khái niệm rất dễ hiểu. Nếu tồn tại đường đi $u \rightarrow v$, ta nói u, v liên thông (vẫn dùng ký hiệu $u \rightarrow v$ để biểu thị u, v liên thông). Đây là một quan hệ tương đương. Tiếp theo ta định nghĩa một số khái niệm sẽ dùng về sau.

Định nghĩa 2.1 (tập cắt đỉnh/cạnh). Với đồ thị vô hướng $G = (V, E)$ và $u, v \in V$: nếu $S \subseteq V$ thỏa $u, v \notin S$ và trong đồ thị con cảm sinh của G theo $V \setminus S$ ta có $u \not\rightarrow v$, thì S được gọi là một tập cắt đỉnh của u, v ; nếu $T \subseteq E$ thỏa trong $G' = (V, E \setminus T)$ ta có $u \not\rightarrow v$, thì T được gọi là một tập cắt cạnh của u, v .

Định nghĩa 2.2 (độ liên thông đỉnh/cạnh). Với đồ thị vô hướng $G = (V, E)$ và $u, v \in V, u \neq v$, nếu mọi tập cắt đỉnh của u, v đều có kích thước không nhỏ hơn s , ta nói u, v là s -liên thông đỉnh; nếu mọi tập cắt cạnh của u, v đều có kích thước không nhỏ hơn t , ta nói u, v là t -liên thông cạnh. Độ liên thông đỉnh/cạnh giữa u, v là giá trị nhỏ nhất của s, t ở trên. Độ liên thông đỉnh/cạnh của G là giá trị nhỏ nhất trên mọi cặp đỉnh.

Ví dụ, 1-liên thông đỉnh và 1-liên thông cạnh là tương đương; cả hai đều tương đương với tính liên thông theo nghĩa thông thường.

Định lý 2.1 (Menger). Với đồ thị vô hướng $G = (V, E)$ và $u, v \in V, u \neq v$:

- u, v là k -liên thông cạnh khi và chỉ khi tồn tại k đường đi $u \rightarrow v$ đôi một không chung cạnh;
- u, v là k -liên thông đỉnh khi và chỉ khi tồn tại k đường đi $u \rightarrow v$ đôi một không chung đỉnh ngoài hai đầu mút.

Đây thực chất là một phiên bản đặc biệt của định lý luồng cực đại cắt cực tiểu. Vì chứng minh không liên quan nhiều tới bài viết này nên được lược bỏ. Cần chú ý rằng trong định lý Menger ta không yêu cầu các đường đi phải khác nhau. Đồ thị đầy đủ K_2 gồm hai đỉnh là k -liên thông đỉnh với mọi k , nhưng trong đó chỉ tồn tại một đường đi đơn.

3.2.1 Cây DFS

Trước hết ta xét loại liên thông đơn giản nhất, tức 1-liên thông. Với một đồ thị vô hướng liên thông G , luôn tìm được một đồ thị con không chu trình T chứa tất cả các đỉnh. Ta gọi T như vậy là cây khung của G .

Định nghĩa 2.3 (thứ tự DFS, cây DFS). Trên đồ thị vô hướng liên thông $G = (V, E)$, thực hiện tìm kiếm theo chiều sâu từ đỉnh xuất phát $r \in V$. Nếu đỉnh x là đỉnh thứ p được thăm, gọi p là thứ tự DFS của x , ký hiệu $\text{dfn}_x = p$. Nếu $x \neq r$, nối một cạnh giữa x và đỉnh y lần đầu thăm được x ; đồ thị tạo thành được gọi là một cây DFS gốc r , ký hiệu T .

Đồ thị T ở trên có $n - 1$ cạnh, và có thể chứng minh bằng quy nạp rằng T liên thông, nên T thật sự là một cây khung của G . Thông thường ta xem nó là cây có gốc r ; các cạnh trên T gọi là cạnh cây, các cạnh còn lại gọi là cạnh không thuộc cây.

Cần lưu ý rằng thứ tự DFS và cây DFS tương ứng một-một. Cùng một r có thể có nhiều thứ tự DFS và nhiều cây DFS khác nhau.

Tính chất 2.1.1. Thứ tự DFS của toàn bộ các đỉnh tạo thành một hoán vị của $1, \dots, n$. Với mọi đỉnh x , tập thứ tự DFS của tất cả đỉnh trong cây con gốc x trên cây DFS tạo thành một đoạn liên tiếp bắt đầu từ dfn_x .

Tính chất 2.1.2. Mọi cạnh không thuộc cây đều nối một cặp đỉnh tổ tiên–hậu duệ trên cây DFS. Các cạnh như vậy gọi là cạnh ngược lên tổ tiên.

Chứng minh chỉ cần xét tính chất của quá trình DFS. Tính chất 2.1.2 là tính chất cốt lõi của cây DFS; đây cũng là lý do nhiều bài toán liên quan tới đồ thị liên thông được xét bằng cây DFS.

Ví dụ 2.1.1 (Ehab’s Last Theorem¹). Với đồ thị vô hướng liên thông G có n đỉnh, ít nhất một trong hai mệnh đề sau đúng:

¹CF 1325 F

1. G có một chu trình đơn độ dài không nhỏ hơn $\sqrt{n}$.
2. G có một tập độc lập kích thước không nhỏ hơn $\sqrt{n}$.

Chứng minh. Đặt $p = \lceil \sqrt{n} \rceil$. Ta đưa ra một chứng minh mang tính xây dựng. Trước hết dựng một cây DFS gốc 1 của G , và ký hiệu d_x là độ sâu của x trên cây DFS.

Xét một cạnh không thuộc cây bất kỳ $x \rightarrow y$, trong đó x là hậu duệ. Nếu $d_x - d_y \geq p - 1$, ta đã tìm được một chu trình đơn có độ dài ít nhất p , gồm đường đi trên cây $x \rightarrow y$ cộng với cạnh không thuộc cây $x \rightarrow y$.

Giả sử không tồn tại chu trình như vậy. Khi đó với mọi cạnh không thuộc cây $x \rightarrow y$ ta có $|d_x - d_y| < p - 1$. Ta có thể xây dựng một tập độc lập kích thước ít nhất p bằng quá trình sau:

- Định nghĩa tập đỉnh S , ban đầu rỗng.
- Chọn một đỉnh chưa bị đánh dấu có độ sâu lớn nhất hiện tại, gọi là x . Đặt $S \leftarrow S \cup \{x\}$, rồi đánh dấu các tổ tiên bậc $0, 1, 2, \dots, p - 2$ của x .
- Lặp lại cho tới khi mọi đỉnh đều bị đánh dấu. Khi đó S là tập độc lập cần tìm.

Trong mỗi vòng, nhiều nhất $p - 1$ đỉnh bị đánh dấu, nên $|S| \geq \frac{n}{p-1} > p - 1$. Đồng thời, đỉnh x được chọn chỉ có thể có cạnh tới những đỉnh trong cây con của nó hoặc tới các tổ tiên bậc $0, 1, \dots, p - 2$. Cách chọn x theo độ sâu lớn nhất bảo đảm các đỉnh trong cây con đã bị đánh dấu, và các tổ tiên bậc $0, 1, \dots, p - 2$ cũng bị đánh dấu. Vì vậy ở mọi thời điểm, mọi đỉnh có cạnh nối với một đỉnh trong S đều đã bị đánh dấu, suy ra S thật sự là tập độc lập. Mệnh đề được chứng minh. ■

Với đồ thị vô hướng không liên thông, ta có thể chọn riêng một đỉnh gốc trong mỗi thành phần liên thông và dựng cây DFS, tạo thành một rừng DFS. Các khái niệm như thứ tự DFS vẫn dùng được, và các tính chất trên vẫn áp dụng. Chẳng hạn ví dụ 2.1.1 thật ra cũng đúng với đồ thị không liên thông; chứng minh tương tự trường hợp liên thông.

3.2.2 Song liên thông

Tiếp theo ta nghiên cứu tính 2-liên thông, phức tạp hơn một chút so với 1-liên thông, còn gọi là song liên thông.

Vì song liên thông là nội dung khá quen thuộc trong OI, phần cơ sở ở đây sẽ được trình bày ngắn gọn hơn, nhưng vẫn nêu đầy đủ mọi tính chất cần dùng. Nội dung này cũng được mô tả toàn diện trong tài liệu tham khảo [1].

Song liên thông cạnh. Do song liên thông cạnh trực quan hơn song liên thông đỉnh, ta giới thiệu nó trước.

Một cặp đỉnh song liên thông cạnh chắc chắn là 1-liên thông, nên các thành phần liên thông khác nhau độc lập với nhau. Vì vậy trước tiên xét trường hợp G liên thông. Điều đầu tiên cần nói là song liên thông cạnh (có thể tổng quát lên k -liên thông cạnh bất kỳ) là một quan hệ tương đương trên tập đỉnh V . Nói cách khác, ta có thể phân hoạch V theo quan hệ song liên thông cạnh.

Định nghĩa 2.4 (cạnh cắt). Với đồ thị vô hướng liên thông $G = (V, E)$, nếu $e \in E$ thỏa $G' = (V, E \setminus \{e\})$ không liên thông, thì e được gọi là một cạnh cắt hay cầu. Với đồ thị vô hướng không nhất thiết liên thông, mọi cạnh cắt của một thành phần liên thông bất kỳ đều được gọi là cạnh cắt của đồ thị.

Một mô tả tương đương là: nếu $\{e\}$ là tập cắt cạnh của một cặp đỉnh (u, v) , thì e là cạnh cắt.

Bổ đề 2.2.1. Trong đồ thị vô hướng liên thông G , mọi cạnh cắt đều nằm trên cây DFS. Giả sử một cạnh cắt là $e = x \rightarrow y$, trong đó x là đỉnh con. Sau khi xóa e , đồ thị tách thành hai thành phần liên thông: cây con của x và phần bù của cây con đó.

Bổ đề 2.2.2. Gọi E' là tập các cạnh cắt của đồ thị vô hướng liên thông $G = (V, E)$, và $G' = (V, E \setminus E')$. Khi đó với mọi $u, v \in V$, $u \neq v$, hai đỉnh u, v song liên thông cạnh trong G khi và chỉ khi u, v liên thông trong G' .

Chứng minh. Nếu u, v liên thông trong G' nhưng không song liên thông cạnh, thì tồn tại một cạnh e sao cho sau khi xóa e , u, v không còn liên thông. Khi đó e tất nhiên là cạnh cắt. Tuy nhiên trong G' đã xóa mọi cạnh cắt, u, v vẫn liên thông, mâu thuẫn. Do đó u, v song liên thông cạnh.

Nếu u, v không liên thông trong G' , xét đường đi trên cây $u \rightarrow v$. Trên đường đi này chắc chắn tồn tại một cạnh e là cạnh cắt (nếu không thì u, v đã liên thông). Theo bổ đề 2.2.1, sau khi xóa cạnh này, u, v không nằm trong cùng một thành phần liên thông, nên u, v không song liên thông cạnh. ■

Định nghĩa 2.5 (thành phần song liên thông cạnh). Các đỉnh của đồ thị vô hướng G có thể được phân hoạch thành vài lớp tương đương theo quan hệ song liên thông cạnh. Đồ thị con cảm sinh bởi mỗi lớp tương đương được gọi là một thành phần song liên thông cạnh của G .

Sự tồn tại của thành phần song liên thông cạnh được bảo đảm bởi bổ đề 2.2.2: chỉ cần xóa mọi cạnh cắt của G để được G' , khi đó mọi thành phần liên thông của G' chính là các thành phần song liên thông cạnh của G .

Ta có thể dựa vào bổ đề 2.2.2 để tìm tất cả thành phần song liên thông cạnh; với một số kỹ thuật cấu trúc dữ liệu, độ phức tạp thời gian có thể đạt $O(n + m)$. Tuy nhiên thuật toán Tarjan cũng có cùng độ phức tạp và gọn hơn.

Xét khi nào một cạnh trở thành cạnh cắt. Gọi cạnh cây DFS là $e = (x, y)$, trong đó x là đỉnh con. Khi đó e là cạnh cắt khi và chỉ khi không tồn tại cạnh từ một đỉnh trong cây con của x tới một tổ tiên của x (không tính x). Chú ý rằng mọi cạnh không thuộc cây đều là cạnh ngược lên tổ tiên. Vì vậy ta có thể tính low_x : thứ tự DFS nhỏ nhất của một đỉnh có thể tới được từ một đỉnh bất kỳ trong cây con của x bằng cách đi qua nhiều nhất một cạnh không thuộc cây. Giá trị này được tính bằng quy hoạch động trên cây:

$$\text{low}_x = \min \left(\text{dfn}_x, \min_{y \in \text{Son}(x)} \text{low}_y, \min_{(x,y) \in E} \text{dfn}_y \right).$$

Những đỉnh thỏa $\text{low}_x = \text{dfn}_x$ là đỉnh nông nhất của một thành phần song liên thông cạnh. Khi trong quá trình DFS phát hiện một đỉnh như vậy, phần chưa bị xóa trong cây con của nó tạo thành một thành phần song liên thông cạnh; sau đó xóa cây con này.

Hình 1: Một đồ thị vô hướng và các thành phần song liên thông cạnh khác nhau của nó.

Song liên thông đỉnh. Song liên thông đỉnh không phải là quan hệ tương đương trên các đỉnh, nhưng ta vẫn có thể bắt đầu nghiên cứu từ cây DFS.

Định nghĩa 2.6 (đỉnh cắt). Với đồ thị vô hướng liên thông $G = (V, E)$, nếu $u \in V$ thỏa đồ thị con cảm sinh của G theo $V \setminus \{u\}$ không liên thông, thì u được gọi là một đỉnh cắt. Với đồ thị vô hướng không nhất thiết liên thông, mọi đỉnh cắt của một thành phần liên thông bất kỳ đều được gọi là đỉnh cắt của đồ thị.

Đỉnh cắt cũng có mô tả tương đương: nếu $\{u\}$ là tập cắt đỉnh của một cặp đỉnh nào đó, thì u là đỉnh cắt.

Nhắc lại mảng low trong thuật toán Tarjan: low_x là thứ tự DFS nhỏ nhất của đỉnh có thể tới được từ một đỉnh bất kỳ trong cây con của x sau khi đi qua nhiều nhất một cạnh không thuộc cây. Ta có thể dùng nó để tìm đỉnh cắt.

Bổ đề 2.2.3. Trong đồ thị vô hướng liên thông $G = (V, E)$, x là đỉnh cắt khi và chỉ khi ít nhất một trong hai điều kiện sau đúng:

1. x là gốc của cây DFS và có ít nhất hai đỉnh con;
2. x không phải gốc của cây DFS, và tồn tại một đỉnh con y thỏa $\text{low}_y \geq \text{dfn}_x$.

Chứng minh. Điều kiện thứ nhất khá hiển nhiên.

Xét điều kiện thứ hai. Nếu tồn tại đỉnh con y như vậy, thì trong cây con của y không có cạnh ngược tới tổ tiên của x (không tính x). Sau khi xóa x , cây con của y không liên thông với bên ngoài và tự tạo thành một thành phần liên thông, nên x chắc chắn là đỉnh cắt.

Ngược lại, nếu đỉnh không phải gốc x là đỉnh cắt, gọi V' là phần bù của cây con x trong V . Sau khi xóa x , tồn tại ít nhất một đỉnh con y của x không liên thông với V' . Điều này nghĩa là không đỉnh nào trong cây con của y có cạnh ngược tới V' , tức $\text{low}_y \geq \text{dfn}_x$. ■

Định nghĩa 2.7 (thành phần song liên thông đỉnh). Trong đồ thị vô hướng $G = (V, E)$, nếu tập đỉnh $V' \subseteq V$ thỏa đồ thị con cảm sinh G' của V' là song liên thông đỉnh, và với mọi $V' \subset V'' \subseteq V$, đồ thị con cảm sinh của V'' không song liên thông đỉnh, thì G' được gọi là một thành phần song liên thông đỉnh của G .

Thuật toán Tarjan có thể dùng để tìm tất cả thành phần song liên thông đỉnh, theo cách tương tự tìm thành phần song liên thông cạnh: thực hiện quy hoạch động trên cây để tính low_x . Nếu với một cặp cha con (x, y) ta có $\text{low}_y \geq \text{dfn}_x$, thì lấy phần chưa bị xóa trong cây con của y , cộng thêm x , để tạo thành một thành phần song liên thông đỉnh. Sau đó xóa phần chưa bị xóa trong cây con của y . Độ phức tạp thời gian là $O(n + m)$.

Dựa trên bổ đề 2.2.3 và chứng minh của nó, không khó để chứng minh tính đúng đắn của thuật toán Tarjan. Tuy nhiên hiện tại ta chưa nắm nhiều tính chất của thành phần song liên thông đỉnh. Trong mục 2.2.1, ta định nghĩa thành phần song liên thông cạnh dựa trên phân hoạch của V' . Vậy thành phần song liên thông đỉnh có hình thức tương tự hay không? Câu trả lời là có, nhưng ta cần chuyển đổi tương nghiên cứu từ đỉnh sang cạnh. Tiếp theo ta đưa ra một cách mô tả thành phần song liên thông đỉnh từ góc nhìn cạnh. Trong phần còn lại của mục này, mặc định đồ thị không có khuyên.

Bổ đề 2.2.4. Trong đồ thị song liên thông đỉnh G , với hai đỉnh khác nhau bất kỳ u, v , tồn tại một chu trình đơn đi qua u, v .

Chứng minh. Đây là hệ quả trực tiếp của định lý Menger. ■

Bổ đề 2.2.5. Trong đồ thị song liên thông đỉnh G , với một đỉnh bất kỳ u và một cạnh bất kỳ e , tồn tại một chu trình đơn đi qua u, e ; với hai cạnh bất kỳ e_1, e_2 , tồn tại một chu trình đơn đi qua e_1, e_2 .

Chứng minh. Tách một cạnh $x \rightarrow y$ thành hai cạnh $x \rightarrow z$ và $z \rightarrow y$. Hiển nhiên đồ thị mới vẫn song liên thông đỉnh, và lúc này cạnh đã được chuyển thành đỉnh. Áp dụng trực tiếp bổ đề 2.2.4 là đủ. ■

Định nghĩa 2.8. Trong đồ thị vô hướng G , nếu hai cạnh e_1, e_2 cùng nằm trên một chu trình đơn, ta nói chúng đồng chu trình.

Bổ đề 2.2.6. Quan hệ đồng chu trình là một quan hệ tương đương; các cạnh trong mỗi thành phần song liên thông đỉnh tạo thành một lớp tương đương theo nghĩa đồng chu trình.

Chứng minh. Trước hết, hai cạnh bất kỳ trong cùng một thành phần song liên thông đỉnh đều đồng chu trình, theo bổ đề 2.2.5.

Với hai cạnh thuộc hai thành phần song liên thông đỉnh khác nhau $e_1 = (x, y)$, $e_2 = (u, v)$, giả sử tồn tại một chu trình đơn đồng thời chứa e_1, e_2 . Khi đó mọi đỉnh trên chu trình này chắc chắn song liên thông đỉnh, mâu thuẫn với việc e_1, e_2 thuộc hai thành phần song liên thông đỉnh khác nhau. ■

Cần chú ý rằng dù trong định nghĩa thành phần song liên thông đỉnh và trong chứng minh trên, ta chưa chứng minh một sự thật cơ bản: mỗi cạnh chỉ thuộc một thành phần song liên thông đỉnh. Thực ra từ định nghĩa hoặc từ thuật toán Tarjan đều có thể dễ dàng suy ra giao của hai thành phần song liên thông đỉnh bất kỳ có kích thước nhiều nhất 1, nên mỗi cạnh đúng là chỉ nằm trong một thành phần song liên thông đỉnh.

Nhờ bổ đề 2.2.6, ta có thể nhìn thành phần song liên thông đỉnh tương tự thành phần song liên thông cạnh; chỉ khác là thành phần song liên thông cạnh là phân hoạch tập đỉnh, còn thành phần song liên thông đỉnh là phân hoạch tập cạnh.

Cây tròn–vuông. Cây tròn–vuông là một cấu trúc dữ liệu dùng để mô tả cấu trúc các thành phần song liên thông đỉnh của đồ thị vô hướng.

Định nghĩa 2.9 (cây tròn–vuông). Với đồ thị vô hướng liên thông $G = (V, E)$ có ít nhất 2 đỉnh, xây một đồ thị vô hướng mới T . Mỗi đỉnh của T tương ứng với một đỉnh của G hoặc một thành phần song liên thông đỉnh của G . Hai đỉnh của T có cạnh khi và chỉ khi chúng lần lượt biểu diễn một đỉnh x của G và một thành phần song liên thông đỉnh $G' = (V', E')$ của G , đồng thời $x \in V'$. Đồ thị T được gọi là cây tròn–vuông của G . Trong T , đỉnh tương ứng với đỉnh của đồ thị gốc gọi là đỉnh tròn; đỉnh tương ứng với thành phần song liên thông đỉnh của đồ thị gốc gọi là đỉnh vuông.

Hình 2: Một đồ thị vô hướng, các thành phần song liên thông đỉnh khác nhau của nó, và cây tròn–vuông.

Xét một đường đi độ dài k là $u \rightarrow v$. Nó lần lượt đi qua các thành phần song liên thông đỉnh chứa từng cạnh của nó, nên tương ứng với một đường đi độ dài $2k$ từ u tới v trên cây tròn–vuông: ngoài các đỉnh trên đường đi gốc, nó còn lần lượt đi qua các đỉnh vuông tương ứng với các thành phần song liên thông đỉnh chứa từng cạnh. Vì vậy cây tròn–vuông của một đồ thị liên thông là liên thông. Đồng thời, cây tròn–vuông chắc chắn không có chu trình, nếu không chu trình đó có thể co thành một thành phần song liên thông đỉnh. Do đó cây tròn–vuông thật sự là một cây. Với đồ thị vô hướng G không nhất thiết liên thông, có thể định nghĩa “rừng tròn–vuông” bằng cách dựng cây tròn–vuông riêng cho từng thành phần liên thông.

Tính chất 2.2.1. Cây tròn–vuông là đồ thị hai phía; hai phía lần lượt là các đỉnh tròn và các đỉnh vuông.

Tính chất 2.2.2. Một đỉnh là đỉnh cắt khi và chỉ khi đỉnh tròn tương ứng của nó trên cây tròn–vuông có bậc lớn hơn 1. Lá trên cây tròn–vuông (kể cả gốc bậc 1) chắc chắn là đỉnh tròn, và chúng chính là toàn bộ các đỉnh không phải đỉnh cắt.

Tính chất 2.2.3. Trong một đồ thị vô hướng có $n \geq 2$ đỉnh, tổng số đỉnh của tất cả thành phần song liên thông đỉnh không vượt quá $2n - 2$.

Chứng minh. Chọn một đỉnh tròn làm gốc trên cây tròn–vuông. Sau đó, với mỗi đỉnh vuông, chỉ định một đỉnh con của nó. Các đỉnh con này đều là đỉnh tròn và chắc chắn không trùng nhau, nên số đỉnh vuông không vượt quá số đỉnh tròn không phải gốc, tức $n - 1$.

Vì vậy tổng số đỉnh của cây tròn–vuông không vượt quá $2n - 1$, tổng số cạnh không vượt quá $2n - 2$. Tổng kích thước của các thành phần song liên thông đỉnh chính là tổng bậc của các đỉnh vuông, tức tổng số cạnh của cây tròn–vuông. Suy ra kết luận. ■

Tính chất 2.2.4. Với hai đỉnh $u, v \in V$ trong đồ thị vô hướng liên thông G , mọi đường đi đơn giữa u và v chắc chắn đi qua, và chỉ đi qua, tất cả thành phần song liên thông đỉnh tương ứng với các đỉnh vuông trên đường đi giữa chúng trong cây tròn–vuông (cụ thể là đi qua các cạnh nằm trong những thành phần đó); đồng thời chắc chắn đi qua mọi đỉnh tròn trên đường đi đó (tức các đỉnh tương ứng trong đồ thị gốc).

Cây tròn–vuông là một cấu trúc rất quan trọng trong OI và có nhiều ứng dụng. Dưới đây là vài ví dụ.

Ví dụ 2.2.1 (xương rồng). Đồ thị vô hướng liên thông mà mỗi cạnh nằm trên nhiều nhất một chu trình đơn được gọi là xương rồng theo cạnh. Nhiều thao tác vốn làm được trên cây có thể được chuyển sang xương rồng thông qua cây tròn–vuông.

Xét thuật toán Tarjan trên xương rồng. Ta nhận được một cây DFS, và mỗi cạnh không thuộc cây tương ứng với một đường đi trên cây; các đường đi này đôi một không chung cạnh. Mỗi cạnh không thuộc cây tương ứng với một chu trình đơn. Vì vậy quá trình Tarjan trực tiếp giúp ta tìm mọi chu trình, điều này rất tiện. Ví dụ khi cần DP trên xương rồng, ta có thể làm từ dưới lên trên cây tròn–vuông (hoặc trên cây DFS): mỗi khi tìm được một chu trình thì DP trên chu trình, nếu không thì DP trên cây.

Khi cần sửa đổi/truy vấn đường đi trên xương rồng, thông thường phải chuyển đường đi trên xương rồng thành đường đi trên cây tròn–vuông, rồi qua một số thảo luận chuyển bài toán thành bài toán trên cây. Tuy nhiên loại bài này thường khó cài đặt, và hiện không thường gặp trong OI.

Ví dụ 2.2.2 (Địa địa thiết thiết²). Cho đồ thị vô hướng liên thông $G = (V, E)$, các cạnh có hai màu đen và trắng. Hãy tính số cặp đỉnh không thứ tự (x, y) sao cho tồn tại một đường đi đơn $x \rightarrow y$ có cả cạnh đen lẫn cạnh trắng.

Ta chỉ cần đếm các cặp (x, y) không thỏa điều kiện. Có ba trường hợp:

1. mọi đường đi đơn $x \rightarrow y$ chỉ đi qua cạnh đen;
2. mọi đường đi đơn $x \rightarrow y$ chỉ đi qua cạnh trắng;
3. tồn tại riêng một đường đi đơn $x \rightarrow y$ chỉ gồm cạnh đen và một đường đi đơn chỉ gồm cạnh trắng, nhưng không có đường đi đơn nào đồng thời đi qua hai màu cạnh.

Trước hết xét trường hợp 1. Theo tính chất 2.2.4 của cây tròn–vuông, ta biết chỉ cần xét các cạnh trong mọi đỉnh vuông nằm trên đường đi $x \rightarrow y$ của cây tròn–vuông, vì các cạnh khác không thể xuất hiện trên đường đi đơn nào từ x tới y .

Gọi một đỉnh vuông là thuần đen khi và chỉ khi mọi cạnh trong thành phần song liên thông đỉnh tương ứng đều là đen; thuần trắng được định nghĩa tương tự. Ta khẳng định trường hợp 1 xảy ra khi và chỉ khi mọi đỉnh vuông trên đường đi $x \rightarrow y$ của cây tròn–vuông đều thuần đen.

Bổ đề phụ: trong đồ thị song liên thông đỉnh, với hai đỉnh cho trước x, y ($x \neq y$) và một cạnh e , tồn tại một đường đi đơn $x \rightarrow y$ đi qua e .

Chứng minh bổ đề phụ chỉ cần nhắc lại bổ đề 2.2.5. Trước hết tìm một chu trình C chứa x, e , sau đó chọn một đường đi đơn P từ y tới x sao cho tập đỉnh của P và C giao nhau nhiều hơn một đỉnh (chú ý x

²Luogu P8456, SWTR-8

chắc chắn nằm trong giao, nên luôn tìm được P như vậy; nếu không, x là đỉnh cắt). Gọi u là đỉnh gần y nhất trên P trong giao của hai tập đỉnh. Khi đó chọn đường đi trên chu trình từ x tới u có chứa e , ghép với tiền tố $u \rightarrow y$ của P , ta được một đường đi đơn $x \rightarrow y$ đi qua e .

Vì vậy nếu tồn tại một đỉnh vuông trên đường đi không thuần đen, khi đi tới thành phần song liên thông đỉnh đó ta có thể tùy ý đi qua một cạnh trắng trong đó, mâu thuẫn với trường hợp 1. Ngược lại, nếu mọi đỉnh vuông trên đường đi đều thuần đen, hiển nhiên mọi đường đi đơn từ x tới y đều thuần đen. Khẳng định được chứng minh. Trường hợp 2 tương tự trường hợp 1.

Trường hợp 3 nhìn qua phức tạp hơn một chút. Trước hết cần một bổ đề.

Bổ đề phụ: nếu (x, y) thỏa trường hợp 3, thì với hai đường đi đơn bất kỳ P_1, P_2 từ x tới y lần lượt thuần trắng và thuần đen, chúng không chung đỉnh nào ngoài hai đầu mút.

Chứng minh bổ đề không khó: giả sử hai đường đi còn giao nhau ở một đỉnh khác ngoài đầu mút, thì có thể lấy một tiền tố và một hậu tố để tạo thành một đường đi mới; đường đi mới không thuần đen cũng không thuần trắng, mâu thuẫn với mô tả của trường hợp 3.

Bổ đề này cho biết x, y chắc chắn nằm trong cùng một thành phần song liên thông đỉnh, nếu không mọi đường đi giữa chúng đều đi qua một đỉnh cắt nào đó. Ngoài ra, giả sử trong thành phần song liên thông đỉnh chứa x, y còn có một đỉnh $u \neq x, y$ kề với hai cạnh e_1, e_2 lần lượt đen và trắng, thì x, y chắc chắn không thỏa trường hợp 3. Lý do là ta có thể tìm hai đường đi đơn $x \rightarrow y$ lần lượt đi qua e_1, e_2 , và chúng giao nhau tại u , mâu thuẫn với bổ đề.

Từ phân tích trên, trong mỗi thành phần song liên thông đỉnh có nhiều nhất một cặp (x, y) thỏa trường hợp 3; chỉ cần tìm các đỉnh kề đồng thời cạnh đen và cạnh trắng.

Sau khi phân tích ba trường hợp, phần còn lại chỉ là tính số lượng các cặp đỉnh này; phần đó khá đơn giản nên lược bỏ.

Ví dụ 2.2.3 (Tom & Jerry³). Trên một đồ thị vô hướng liên thông G , Tom đuổi Jerry. Mỗi lượt Jerry đi trước, Tom đi sau. Trong mỗi lần đi, Jerry có thể đi qua tùy ý nhiều cạnh nhưng không được đi qua đỉnh Tom đang đứng; Tom mỗi lần chỉ được đi qua nhiều nhất một cạnh. Nếu sau khi đi, Tom đứng tại vị trí của Jerry thì Tom thắng. Có q truy vấn (a, b) : khi ban đầu Tom và Jerry lần lượt đứng tại a, b , hỏi Tom có thể thắng trong thời gian hữu hạn hay không.

Định nghĩa cặp đỉnh (x, y) là tốt khi và chỉ khi mọi cặp đỉnh tròn kề nhau trên đường đi $x \rightarrow y$ của cây tròn–vuông cũng kề nhau trong đồ thị gốc G .

Kết luận chính: Tom thắng khi và chỉ khi ít nhất một trong hai điều kiện sau đúng:

1. tồn tại một đỉnh u sao cho với mọi x , cặp (u, x) là tốt;
2. với mọi đỉnh x trong thành phần liên thông chứa b sau khi xóa a , cặp (a, x) là tốt.

Trước hết chứng minh tính đủ. Hai điều kiện thực ra tương tự nhau; lấy điều kiện 2 làm ví dụ. Mỗi lần Tom chỉ cần tiến một bước về phía Jerry theo đường đi trên cây tròn–vuông (tính khả thi ở đây được điều kiện 2 bảo đảm). Từ tính chất của cây tròn–vuông, Jerry luôn chỉ có thể di chuyển trong một nhánh của đỉnh Tom đang đứng; do đó cuối cùng Tom và Jerry sẽ gặp nhau, Tom thắng.

Xét tính cần. Giả sử cả hai điều kiện đều không thỏa. Khi đó bước đầu tiên Jerry có thể đi tới một đỉnh x sao cho (a, x) không tốt, rồi đứng yên chờ tới khi Tom đi tới một đỉnh y không nằm trên đường đi cây $a \rightarrow x$. Vì điều kiện 1 không đúng, trong toàn đồ thị chắc chắn tồn tại một đỉnh z sao cho (y, z) không tốt. Nếu đường đi cây $z \rightarrow x$ không đi qua x , lúc này Jerry đi thẳng từ x tới z ; nếu không, Jerry có thể đi tới z ở bước ngay trước khi Tom đi tới y . Tóm lại, ta lại trở về tình huống cặp đỉnh gồm vị trí của Tom và vị trí của Jerry không tốt; vì vậy Tom sẽ không bao giờ bắt được Jerry.

³Luogu P7353; bài này do tác giả ra cho bài tập trại tình anh IOI 2021.

Sau khi có kết luận chính, thêm DP trên cây tròn–vuông là giải được bài này. Phần sau không liên quan tới bài viết nên lược bỏ.

Phân rã tai. Phân rã tai là một cách mô tả khác của đồ thị song liên thông, và có thể cung cấp cho ta một góc nhìn khác trong một số bài toán.

Định nghĩa 2.10 (tai, tai mở). Trong đồ thị vô hướng $G = (V, E)$ có một đồ thị con $G' = (V', E')$. Nếu một đường đi đơn hoặc chu trình đơn $P : x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_k$ thỏa $x_1, x_k \in V'$ và $x_2, \dots, x_{k-1} \notin V'$, thì P được gọi là một tai của G đối với G' . Nếu P là đường đi đơn, P được gọi là tai mở của G đối với G' .

Định nghĩa 2.11 (phân rã tai, phân rã tai mở). Với đồ thị vô hướng liên thông G , nếu dãy đồ thị liên thông $(G_0, G_1, \dots, G_k)$ thỏa:

1. G_0 là một chu trình đơn (có thể chỉ có một đỉnh), $G_k = G$;
2. G_{i-1} là đồ thị con của G_i ;
3. đặt $G_i = (V_i, E_i)$, khi đó $E_i \setminus E_{i-1}$ tạo thành một tai (tai mở) của G_{i-1} ,

thì $(G_0, G_1, \dots, G_k)$ được gọi là một phân rã tai (phân rã tai mở) của G .

Định lý 2.2. Đồ thị vô hướng liên thông G có phân rã tai khi và chỉ khi G song liên thông cạnh.

Chứng minh. Trước hết chứng minh tính cần. Nếu G có phân rã tai $(G_0, \dots, G_k)$, thì vì G_0 là chu trình đơn, G_0 chắc chắn song liên thông cạnh. Nếu G_i song liên thông cạnh, thì G_{i+1} thu được bằng cách thêm một đường đi vào G_i cũng song liên thông cạnh (vì không cạnh nào có thể là cạnh cắt). Quy nạp suy ra $G = G_k$ song liên thông cạnh.

Tiếp theo chứng minh tính đủ. Nếu $G = (V, E)$ song liên thông cạnh và không có cạnh nào, kết luận hiển nhiên. Nếu không, dựng một cây DFS gốc 1, rồi lấy một phân rã tai của G theo cách sau:

1. Tìm một cạnh không thuộc cây có đầu mút là 1, viết là $1 \rightarrow x$. Cho G_0 là chu trình đơn gồm đường đi trên cây $1 \rightarrow x$ cộng với cạnh này.
2. Giả sử hiện đã sinh được G_i . Nếu tập đỉnh của G_i chưa phải V , tìm một đỉnh x chưa thuộc G_i sao cho cha y của x thuộc G_i . Do song liên thông cạnh, tồn tại một cạnh ngược có hai đầu lần lượt là một tổ tiên u của y và một hậu duệ v của x . Ta chọn đường đi trên cây $y \rightarrow v$ hợp với cạnh $v \rightarrow u$; đây là một tai của G_i . Cho G_{i+1} bằng G_i cộng tai này.
3. Lặp lại cho tới khi tập đỉnh của G_i là V . Nếu khi đó còn cạnh nào của G chưa được thêm vào G_i , thêm từng cạnh riêng lẻ như một tai.

Ở bước thứ hai, ta luôn bảo đảm tập đỉnh của G_i là một khối liên thông trên cây chứa 1, nên luôn tìm được x như vậy. ■

Ví dụ 2.2.4 (Quare⁴). Cho đồ thị vô hướng song liên thông cạnh G có trọng số cạnh. Hãy tìm tổng trọng số nhỏ nhất của một đồ thị con song liên thông cạnh chứa mọi đỉnh.

Xét trực tiếp hình dạng đồ thị song liên thông cạnh là khá khó, nhưng từ góc nhìn phân rã tai thì dễ xử lý hơn. Gọi $f(S)$ là tổng trọng số nhỏ nhất để nối các đỉnh trong tập S thành một đồ thị song liên thông cạnh. Khi đó chuyển trạng thái có dạng $f(S) + E(S, T \setminus S) \rightarrow f(T)$, trong đó $E(S, R)$ là tổng trọng số nhỏ nhất của một tai có hai đầu mút thuộc S và các đỉnh còn lại thuộc R . Có thể tính $E(S, R)$ bằng

⁴SNOI 2013

cách liệt kê đỉnh cuối cùng ngoài hai đầu mút trên tai; độ phức tạp $O(2^n \text{ Poly}(n))$. Tuy nhiên khi tính mảng f , ta cần liệt kê tập con, nên tổng độ phức tạp là $O(3^n \text{ Poly}(n))$.

Có thể tối ưu điều này. Ta không tính $E(S, R)$, mà trực tiếp liệt kê thứ tự các đỉnh trên tai trong quá trình tính mảng f : mỗi lần chỉ thêm một đỉnh ở cuối tai, thay vì thêm cả tai. Cụ thể, đặt $f(S, i, j)$ là giá trị khi đã nối tập đỉnh S thành một đồ thị song liên thông cạnh, hiện đang thêm một tai, tai này đã kéo dài từ một đầu tới i , và quy định đầu còn lại của tai là $j \in S$. Khi chuyển trạng thái chỉ cần liệt kê đỉnh kế tiếp sau i trên tai. Độ phức tạp là $O(2^n \text{ Poly}(n))$.

Định lý 2.3. Đồ thị vô hướng liên thông không khuyên G có ít nhất ba đỉnh có phân rã tai mở khi và chỉ khi G song liên thông đỉnh.

Chứng minh. Chứng minh nhìn chung tương tự định lý 2.2.

Trước hết chứng minh tính cần. Nếu G có phân rã tai mở $(G_0, \dots, G_k)$, thì vì G_0 là chu trình đơn, G_0 chắc chắn song liên thông đỉnh. Nếu G_i song liên thông đỉnh, thì G_{i+1} thu được bằng cách thêm một đường đi vào G_i cũng song liên thông đỉnh. Quy nạp suy ra $G = G_k$ song liên thông đỉnh.

Tiếp theo chứng minh tính đủ. Nếu $G = (V, E)$ song liên thông đỉnh, dựng một cây DFS gốc 1, rồi lấy một phân rã tai mở của G như sau:

1. Tìm một cạnh không thuộc cây có đầu mút là 1, viết là $1 \rightarrow x$. Cho G_0 là chu trình đơn gồm đường đi trên cây $1 \rightarrow x$ cộng với cạnh này.
2. Giả sử hiện đã sinh được G_i . Nếu tập đỉnh của G_i chưa phải V , tìm một đỉnh x chưa thuộc G_i sao cho cha y của x thuộc G_i . Do song liên thông đỉnh, tồn tại một cạnh ngược có hai đầu lần lượt là một tổ tiên u của cha của y và một hậu duệ v của x . Ta chọn đường đi trên cây $y \rightarrow v$ hợp với cạnh $v \rightarrow u$; đây là một tai mở của G_i . Cho G_{i+1} bằng G_i cộng tai mở này.
3. Lặp lại cho tới khi tập đỉnh của G_i là V . Nếu khi đó còn cạnh nào của G chưa được thêm vào G_i , thêm từng cạnh riêng lẻ như một tai mở (ở đây dùng điều kiện không có khuyên).

Ở bước thứ hai, ta luôn bảo đảm tập đỉnh của G_i là một khối liên thông trên cây chứa 1, nên luôn tìm được x như vậy. ■

Ví dụ 2.2.5 (định hướng lưỡng cực). Cho đồ thị vô hướng $G = (V, E)$ và hai đỉnh khác nhau s, t . Bốn mệnh đề sau tương đương:

1. sau khi thêm cạnh vô hướng $s \rightarrow t$, G song liên thông đỉnh;
2. trong cây tròn–vuông của G , mọi đỉnh vuông tạo thành một dây chuyền, và $s \rightarrow t$ là một đường kính của cây tròn–vuông;
3. tồn tại một cách định hướng các cạnh của G để thu được một đồ thị có hướng không chu trình, trong đó s có bậc vào bằng 0, t có bậc ra bằng 0, và mọi đỉnh còn lại đều có bậc vào và bậc ra khác 0;
4. tồn tại một hoán vị các đỉnh $p_1, p_2, \dots, p_n$ sao cho $p_1 = s, p_n = t$, và đồ thị con cảm sinh bởi mọi tiền tố cũng như mọi hậu tố đều liên thông.

Chứng minh. Trước hết chứng minh mệnh đề 1,2 tương đương và mệnh đề 3,4 tương đương; sau đó chứng minh hai cặp mệnh đề này cũng tương đương.

Sự tương đương của mệnh đề 1 và 2 khá hiển nhiên.

Xét mệnh đề 3 và 4. Nếu mệnh đề 3 đúng, lấy một thứ tự tô pô bất kỳ của đồ thị đã định hướng làm hoán vị p trong mệnh đề 4. Theo mô tả của mệnh đề 3, ngoài s , mỗi đỉnh đều có một láng giềng đứng trước nó trong thứ tự tô pô. Vì vậy với mọi p_i , tồn tại một láng giềng trong $p_1, \dots, p_{i-1}$; từ tính liên thông

của đồ thị con cảm sinh bởi $p_1, \dots, p_{i-1}$ suy ra tính liên thông của đồ thị con cảm sinh bởi $p_1, \dots, p_i$. Quy nạp được mọi tiền tố liên thông; mọi hậu tố cũng tương tự. Nếu mệnh đề 4 đúng, định hướng mỗi cạnh từ đỉnh đứng trước trong p tới đỉnh đứng sau trong p , để kiểm tra mệnh đề 3 đúng.

Tiếp theo chứng minh mệnh đề 1 suy ra mệnh đề 3. Sau khi nối s, t , lấy một phân rã tai mở $(G_0, \dots, G_k)$ của G sao cho G_0 chứa cạnh $s \rightarrow t$ (nhắc lại chứng minh định lý 2.3, ta có thể làm được điều này trong quá trình dựng). G_0 là một chu trình đơn, nên hiển nhiên có thể định hướng mọi cạnh ngoài $s \rightarrow t$ từ s tới t . Tiếp tục quy nạp: nếu G_i đã định hướng xong, nay thêm một tai mở có hai đầu u, v để được G_{i+1} . Nếu trong định hướng của G_i , u tới được v , định hướng tai từ u tới v ; ngược lại định hướng tai từ v tới u . Điều này không phá vỡ tính không chu trình, và mọi đỉnh ngoài s, t khi được thêm vào đều có bậc vào và bậc ra bằng 1. Vì vậy định hướng cuối cùng của $G = G_k$ thỏa mệnh đề 3.

Cuối cùng chứng minh mệnh đề 4 suy ra mệnh đề 1. Giả sử mệnh đề 1 không đúng. Khi đó tồn tại một đỉnh cắt u sao cho sau khi xóa u , hai đỉnh s, t nằm trong cùng một thành phần liên thông; do đó chắc chắn còn một thành phần liên thông S không chứa s, t . Gọi x, y lần lượt là đỉnh xuất hiện sớm nhất và muộn nhất trong hoán vị p trong tập $S \cup \{u\}$. Ít nhất một trong hai đỉnh x, y không phải u . Không mất tính tổng quát, giả sử $x \neq u$. Xét đồ thị con cảm sinh bởi tiền tố $p_1, \dots, p_i = x$. Vì u không nằm trong đó, còn s, x đều nằm trong đó, đồ thị này chắc chắn không liên thông, mâu thuẫn với mệnh đề 4. Do đó mệnh đề 4 suy ra mệnh đề 1.

Kết hợp các phần trên, ta chứng minh được bốn mệnh đề tương đương, đồng thời đã đưa ra cách dựng từ mệnh đề 1 tới mệnh đề 3,4. ■

Hình 3: Một đồ thị vô hướng, phân rã tai mở sau khi nối s, t , và định hướng lưỡng cực.

3.2.3 Tập cắt và tương đương theo lát cắt

Không gian cắt và không gian chu trình. Trước đó ta đã định nghĩa tập cắt đỉnh và tập cắt cạnh giữa hai điểm, nhưng tập cắt trong phần này có nghĩa hơi khác: nó được định nghĩa trên một phân hoạch hai phần của tập đỉnh V .

Định nghĩa 2.12 (tập cắt). Trong đồ thị vô hướng $G = (V, E)$, với một phân hoạch hai phần $V = V_1 \sqcup V_2$, định nghĩa tập cắt giữa V_1, V_2 là

$$C(V_1, V_2) = \{e \in E \mid e = (x, y), x \in V_1, y \in V_2\}.$$

Trong bài viết này cũng viết gọn là $C(V_1)$ hoặc $C(V_2)$.

Có thể thấy, nếu G không liên thông thì một tập cắt của G có thể tách thành hợp các tập cắt của từng thành phần liên thông. Vì vậy về cơ bản ta chỉ cần xét tập cắt của đồ thị liên thông. Đồ thị liên thông đơn giản nhất là cây, nên ta bắt đầu từ tập cắt của cây.

Bổ đề 2.3.1. Với cây $T = (V, E)$, mọi tập con cạnh $E' \subseteq E$ đều là tập cắt, và phân hoạch V_1, V_2 tương ứng là duy nhất.

Chứng minh. Với tập con cạnh bất kỳ $E' \subseteq E$, sau khi xóa các cạnh trong E' , cây tách thành vài thành phần liên thông. Gọi tập các thành phần này là $\mathcal{C}$. Các cạnh trong E' nối các phần tử của $\mathcal{C}$ thành một cây; gọi cây có đỉnh là các phần tử của $\mathcal{C}$ này là T' .

Nếu E' thật sự là tập cắt, thì với mọi $c \in \mathcal{C}$, tất cả đỉnh trong c phải nằm cùng một phía của tập cắt; với hai đỉnh kề nhau $c_1, c_2 \in \mathcal{C}$ trên T' , chúng phải nằm ở hai phía khác nhau của tập cắt.

Do đó ta nhận được phân hoạch duy nhất có thể: một phía gồm tất cả các đỉnh trong những thành phần liên thông ứng với đỉnh độ sâu lẻ trên T' , phía còn lại gồm tất cả các đỉnh trong những thành phần

liên thông ứng với đỉnh độ sâu chẵn. Để kiểm tra E' đúng là tập cắt của phân hoạch này, nên mệnh đề được chứng minh. ■

Bổ đề 2.3.2. Nếu đồ thị liên thông $G = (V, E)$ có cây khung $T = (V, E_0)$, thì với mọi $E'_0 \subseteq E_0$, tồn tại duy nhất một tập cắt E' của G sao cho $E' \cap E_0 = E'_0$, và phân hoạch V_1, V_2 tương ứng là duy nhất.

Chứng minh. Theo bổ đề 2.3.1, từ E'_0 ta trực tiếp thu được duy nhất một cặp V_1, V_2 . Với mọi cạnh không thuộc cây e , nếu hai đầu của e đều nằm trong V_1 hoặc đều nằm trong V_2 thì $e \notin E'$; ngược lại $e \in E'$. Như vậy thu được tập cắt duy nhất thỏa điều kiện. ■

Ta còn có thể phân tích thêm những cạnh không thuộc cây nào nằm trong E' . Xét cạnh không thuộc cây e có hai đầu x, y . Nếu đường đi trên cây $x \rightarrow y$ có số lẻ cạnh thuộc E'_0 , thì trong T' ở chứng minh bổ đề 2.3.1, hai thành phần chứa x, y có độ sâu khác tính chẵn lẻ, nghĩa là chúng nằm ở hai phía của tập cắt, nên $e \in E'$. Ngược lại $e \notin E'$. Nói cách khác, E' chứa đúng các cạnh không thuộc cây bằng qua số lẻ cạnh cây thuộc E'_0 .

Định lý 2.4 (không gian cắt). Trong đồ thị vô hướng $G = (V, E)$, xem mỗi tập cạnh như một vector m chiều trên $\mathbb{F}_2$. Khi đó tập tất cả các tập cắt tạo thành một không gian tuyến tính trên $\mathbb{F}_2$, gọi là không gian cắt. Số chiều của nó là $n - c$, trong đó c là số thành phần liên thông của G .

Chứng minh. Trước hết xét trường hợp đồ thị liên thông.

Để chứng minh các tập cắt tạo thành không gian tuyến tính, chỉ cần chứng minh hiệu đối xứng của hai tập cắt bất kỳ vẫn là tập cắt. Giả sử có hai tập cắt C_1, C_2 , và các tập con của chúng trên cây khung lần lượt là E_1, E_2 . Khi đó tập con trên cây khung của $C_1 \oplus C_2$ là $E_1 \oplus E_2$. Xét một cạnh không thuộc cây e . Nếu nó bằng qua số lẻ cạnh trong $E_1 \oplus E_2$, thì nhờ tính chất bảo toàn chẵn lẻ của hiệu đối xứng, ta có đúng một trong hai điều sau: e bằng qua số lẻ cạnh trong E_1 , hoặc e bằng qua số lẻ cạnh trong E_2 . Điều này tương đương $e \in C_1 \oplus C_2$, nên $C_1 \oplus C_2$ đúng là tập cắt.

Để tính số chiều không gian, chỉ cần đưa ra một cơ sở. Theo bổ đề 2.3.2, với mỗi cạnh trên cây khung e , định nghĩa $f(e)$ là tập cạnh gồm e và các cạnh không thuộc cây bằng qua e . Khi đó mọi $f(e)$ tạo thành một cơ sở của không gian cắt, nên số chiều là $n - 1$.

Khi đồ thị không liên thông, các thành phần liên thông độc lập với nhau; không gian cắt của toàn đồ thị bằng tổng trực tiếp các không gian cắt của từng thành phần liên thông. Cộng số chiều lại, không gian cắt của G có số chiều $n - c$. ■

Trong đồ thị vô hướng còn có một không gian tuyến tính khác trên $\mathbb{F}_2$. Vì cấu trúc của nó đơn giản và trực quan hơn, còn quá trình thảo luận tương tự không gian cắt, ta chỉ nêu kết luận mà không chứng minh.

Định lý 2.5 (không gian chu trình). Trong đồ thị vô hướng $G = (V, E)$, xét mọi đồ thị con $G' = (V, E')$ sao cho bậc của mỗi đỉnh đều chẵn. Xem các E' này như vector m chiều trên $\mathbb{F}_2$, chúng tạo thành một không gian tuyến tính trên $\mathbb{F}_2$, gọi là không gian chu trình của G . Số chiều của nó là $m - n + c$, trong đó c là số thành phần liên thông.

Tương tự không gian cắt, một cơ sở của không gian chu trình được dẫn ra bởi từng cạnh không thuộc cây: với mỗi cạnh không thuộc cây $e = (x, y)$, hợp của e với đường đi trên cây $x \rightarrow y$ tạo thành một chu trình; các chu trình này là một cơ sở của không gian chu trình.

Định lý 2.6. Không gian cắt và không gian chu trình của cùng một đồ thị vô hướng là bù trực giao của nhau.

Chứng minh. Dùng cùng một rừng khung, ta chứng minh các phần tử trong cơ sở của không gian cắt

và cơ sở của không gian chu trình tạo từ rừng khung này đôi một trực giao. Xét một cạnh cây e_1 và một cạnh không thuộc cây $e_2 = (x, y)$; gọi các phần tử cơ sở tương ứng của không gian cắt và không gian chu trình là v_1, v_2 . Nếu e_1 không nằm trên đường đi cây $x \rightarrow y$, thì hai tập cạnh ứng với v_1, v_2 không giao nhau. Nếu e_1 nằm trên đường đi cây $x \rightarrow y$, thì giao của hai tập cạnh ứng với v_1, v_2 chỉ gồm e_1, e_2 . Trong cả hai trường hợp đều có $v_1 \cdot v_2 = 0$.

Đồng thời tổng số chiều của hai không gian bằng m , nên chúng là bù trực giao của nhau. ■

Ví dụ 2.3.1 (phủ chu trình⁵). Cho đồ thị vô hướng đơn G . Với mỗi $i \in [1, m]$, hãy tính số phần tử trong không gian chu trình có tập cạnh kích thước i .

Bài này có hai hướng suy nghĩ chính.

Cách 1: một cách làm trực tiếp. Xem mỗi cạnh $e = (x, y)$ như một vector n chiều trên $\mathbb{F}_2$, chỉ có chiều thứ x, y bằng 1. Khi đó bài toán chuyển thành đếm số tập con i phần tử có tổng bằng không.

Cách 2: tìm một rừng khung của G , rồi xét một cơ sở của không gian chu trình dẫn ra bởi rừng. Liệt kê một phần tử là tổng của x phần tử cơ sở, và chỉ giữ lại các chiều ứng với cạnh cây (vì số cạnh không thuộc cây đã được liệt kê là x). Bài toán chuyển thành đếm trong các tập con x phần tử của cơ sở, có bao nhiêu tập cho tổng có số bit 1 bằng $i - x$.

Cả hai phép chuyển đổi đều có thể xử lý trực tiếp bằng FWT, độ phức tạp $O(n2^n + \text{Poly}(m))$.

Trên cơ sở đó, cách 1 có thể dùng định nghĩa của FWT để chuyển FWT thành một DP nén trạng thái. Nếu có thể tính popcount và lowbit trong $O(1)$, độ phức tạp đạt $O(2^n + \text{Poly}(m))$.

Tối ưu cách 2 cần thêm một số kiến thức nền. Vì khá phức tạp nên không trình bày chi tiết ở đây; độc giả quan tâm có thể tham khảo lời giải CF1336E2. Thực chất, cách 2 đang tìm phân bố số bit 1 của mọi phần tử trong một không gian tuyến tính trên $\mathbb{F}_2$. Kết luận là: nếu ta giải được bài toán này trong không gian bù trực giao, thì có thể trả lời cho không gian ban đầu với chi phí đa thức. Vì vậy ta có thể xét bài toán trên bù trực giao của không gian chu trình, tức không gian cắt. Số chiều của không gian cắt là $n - c$, có thể liệt kê trực tiếp. Độ phức tạp cuối cùng là $O(2^{n/w}m + \text{Poly}(m))$ theo cách ghi của nguồn.

Ứng dụng của tập cắt và tương đương theo lát cắt. Dù đã giới thiệu các tính chất của không gian cắt, tới đây ta vẫn chưa biết cách kiểm tra một tập cạnh có phải tập cắt hay không.

Giả sử trong đồ thị có tổng cộng t cạnh không thuộc cây. Ta xem mỗi cạnh là một vector t chiều trong $\mathbb{F}_2$: cạnh không thuộc cây thứ i ứng với vector chỉ có bit thứ i bằng 1, còn vector ứng với một cạnh cây có bit bằng 1 tại đúng các cạnh không thuộc cây bằng qua cạnh cây đó, và các bit khác bằng 0.

Xét cơ sở của không gian cắt thu được từ rừng khung. Mỗi phần tử cơ sở có dạng một cạnh cây hợp với tất cả cạnh không thuộc cây bằng qua cạnh cây đó. Theo định nghĩa trên, tổng các vector t chiều ứng với những cạnh trong mỗi phần tử cơ sở bằng không.

Ta biết không gian cắt là không gian tuyến tính. Kết hợp định nghĩa vector t chiều của cạnh ở trên, không khó thu được: một tập cạnh là tập cắt khi và chỉ khi tổng các vector t chiều ứng với mọi cạnh trong tập đó bằng không.

Khi cài đặt, ta không thể thật sự thao tác với vector t chiều. Có thể ánh xạ mỗi chiều tới một giá trị trong phạm vi 2^{64} , rồi biểu diễn một vector bằng xor của các giá trị ứng với các bit bằng 1. Nói cách khác, gán một trọng số ngẫu nhiên cho mỗi cạnh không thuộc cây; định nghĩa trọng số của một cạnh cây là xor của trọng số mọi cạnh không thuộc cây bằng qua nó. Nếu xor trọng số cạnh của một tập cạnh bằng không, ta tin rằng nó là tập cắt.

⁵Bài kiểm tra chéo đội tuyển quốc gia Trung Quốc IOI 2023.

Ví dụ 2.3.2 (DZY Loves Chinese II⁶). Cho đồ thị vô hướng $G = (V, E)$. Có q truy vấn, mỗi truy vấn cho một tập cạnh và hỏi sau khi xóa tập cạnh đó, đồ thị còn liên thông hay không. Bảo đảm kích thước tập cạnh không vượt quá 15.

Theo phép chuyển đổi trên, ta cần xác định liệu có tồn tại một tập con không rỗng của tập cạnh đã cho có xor trọng số bằng không hay không. Chỉ cần xây một cơ sở tuyến tính là kiểm tra được.

Chú ý rằng buộc kích thước tập cạnh không vượt quá 15 là quan trọng. Vì trọng số cạnh được đặt trong phạm vi 2^{64} , còn bài này phải xét mọi tập con để kiểm tra có tập con xor bằng không hay không, nên xác suất sai cao hơn nhiều so với kiểm tra một tập đơn có phải tập cắt. Vì vậy chỉ xử lý được phạm vi nhỏ. Nếu kích thước tập cạnh đã cho lớn hơn 64, dù thế nào ta cũng sẽ kết luận là không liên thông; điều đó rõ ràng vô lý.

Trọng số (vector) cạnh định nghĩa theo cách trên sẽ chia mọi cạnh thành vài lớp tương đương, mỗi lớp gồm các cạnh có trọng số bằng nhau. Ta gọi quan hệ tương đương này là tương đương theo lát cắt. Quan hệ này có vài dạng định nghĩa khác nhau.

Bổ đề 2.3.3. Trong đồ thị vô hướng G , với hai cạnh e_1, e_2 , ba mệnh đề sau tương đương:

1. e_1, e_2 tương đương theo lát cắt;
2. e_1, e_2 đều là cạnh cắt; hoặc e_1, e_2 nằm trong cùng một thành phần song liên thông cạnh, và sau khi xóa e_1, e_2 , thành phần song liên thông cạnh đó không còn liên thông;
3. với mọi chu trình, chu trình đó hoặc đồng thời chứa e_1, e_2 , hoặc đồng thời không chứa e_1, e_2 .

Chứng minh. Từ định nghĩa trọng số cạnh suy ra ngay mệnh đề 1 và mệnh đề 2 tương đương.

Nếu mệnh đề 2 đúng, giả sử có một chu trình chứa e_1 nhưng không chứa e_2 . Khi đó e_1, e_2 không phải cạnh cắt. Xóa e_2 xong, thành phần song liên thông cạnh vẫn liên thông, và lúc này vẫn còn một chu trình chứa e_1 , nên xóa tiếp e_1 cũng không ảnh hưởng tới tính liên thông. Điều này nói rằng sau khi xóa e_1, e_2 , thành phần song liên thông cạnh vẫn liên thông, mâu thuẫn. Do đó mệnh đề 3 đúng.

Nếu mệnh đề 3 đúng, giả sử sau khi xóa e_1, e_2 không phải cạnh cắt. Điều đó nghĩa là tồn tại một chu trình chứa e_2 nhưng không chứa e_1 , mâu thuẫn. Do đó mệnh đề 2 đúng. Vậy mệnh đề 2 và mệnh đề 3 tương đương. ■

Ví dụ 2.3.3 (Tours⁷). Cho đồ thị vô hướng có chu trình G . Cần tô màu mỗi cạnh bằng một trong k màu sao cho trên mỗi chu trình đơn, số cạnh của từng màu xuất hiện bằng nhau. Hỏi k lớn nhất là bao nhiêu.

Trực giác cho thấy, vì yêu cầu mọi chu trình đơn có màu phân bố đều, nên với mỗi lớp tương đương theo lát cắt trừ cạnh cắt, màu sắc trong lớp đó cũng phải phân bố đều. Rõ ràng điều này dẫn tới một phương án tô màu khả thi: lấy k là ước chung lớn nhất của kích thước mọi lớp tương đương không phải cạnh cắt, rồi tô màu đều trong từng lớp. Theo mệnh đề 3 của bổ đề 2.3.3, ta biết trên mỗi chu trình, màu sắc thật sự được phân bố đều.

Tiếp theo cần chứng minh k như vậy là lớn nhất, tức chứng minh mỗi lớp tương đương đều phải được tô đều. Dựng một cây DFS. Giả sử có t cạnh không thuộc cây, và C_i là chu trình đơn chỉ chứa cạnh không thuộc cây thứ i . Ta chỉ cần chứng minh mỗi tập $D_1 \cap D_2 \cap \dots \cap D_t$, trong đó mỗi D_i là C_i hoặc phần bù của C_i , nếu giao này không rỗng thì là một lớp tương đương theo lát cắt, và đều có thể biểu diễn thành tổ hợp tuyến tính hệ số thực của các phần tử $\{\oplus_{i \in S} C_i\}$ trong không gian chu trình. Khi có kết luận này, ta có thể chứng minh riêng cho từng màu rằng số lượng là đúng.

⁶BZOJ 3569

⁷ICPC 2015 World Final

Trước hết chứng minh giao của vài C_i , tức $\bigcap_{i \in S} C_i$, có thể biểu diễn bằng tổ hợp tuyến tính của các $\bigoplus_{i \in S} C_i$. Thật vậy:

$$\bigcap_{i \in S} C_i = \frac{1}{2^{|S|-1}} \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|} \bigoplus_{i \in T} C_i.$$

Kết luận này có thể suy ra từ tính chất của FWT. Vì bài viết không giới thiệu riêng FWT, và kết luận này không liên quan tới chủ đề chính, ta không chứng minh ở đây.

Tiếp theo, dùng nguyên lý bao hàm–loại trừ để chuyển từ giao của vài C_i sang giao của các D_i : nếu một D_i là phần bù của C_i , viết nó thành toàn tập trừ C_i , từ đó chuyển thành tổ hợp tuyến tính của 2^s giao của các tập C_i , trong đó s là số i sao cho D_i là phần bù của C_i . Mệnh đề được chứng minh.

Do đó đáp án chính là ước chung lớn nhất của kích thước mọi lớp tương đương theo lát cắt không phải cạnh cắt. Dùng phương pháp ngẫu nhiên ở trên, độ phức tạp thời gian dễ dàng đạt $O((n+m) \log n)$.

3.3 Đồ thị có hướng

Trong đồ thị có hướng, quan hệ liên thông hai chiều không nhất thiết tồn tại. Nếu trong đồ thị có hướng tồn tại đường đi $x \rightarrow y$, ta nói x tới được y .

Biến mọi cạnh có hướng trong một đồ thị có hướng thành cạnh vô hướng, ta thu được đồ thị vô hướng gọi là đồ thị nền của đồ thị có hướng ban đầu. Nếu đồ thị nền liên thông, ta nói đồ thị có hướng ban đầu liên thông yếu.

Với đồ thị có hướng không chu trình, định nghĩa thứ tự tô pô của nó: thứ tự tô pô là một hoán vị các đỉnh sao cho nếu x tới được y , thì x phải đứng trước y trong hoán vị.

3.3.1 Bài toán khả đạt

Bài toán khả đạt có thể chia thành hai loại tùy theo yêu cầu là quan hệ khả đạt giữa một cặp/một số cặp đỉnh cho trước, hay quan hệ khả đạt giữa mọi cặp đỉnh. Loại sau gọi là bài toán bao đóng bắc cầu.

Bao đóng bắc cầu tĩnh. Trong toán rời rạc, bao đóng của một quan hệ hai ngôi R là quan hệ nhỏ nhất R' sao cho $R \subseteq R'$ và R' thỏa một tính chất nào đó. Bao đóng bắc cầu là R' nhỏ nhất thỏa tính bắc cầu. Rõ ràng với đồ thị có hướng G , trước hết bổ sung quan hệ cạnh R thành phản xạ (tức bổ sung khuyên), rồi bao đóng bắc cầu của nó chính là quan hệ khả đạt R' .

Thuật toán Floyd là thuật toán bao đóng bắc cầu thường dùng nhất. Đặt quan hệ R'_k là quan hệ khả đạt khi chỉ được đi qua các đỉnh đánh số $1, 2, \dots, k$. Bằng cách liệt kê các đường đi qua k , ta có được công thức truy hồi từ R'_{k-1} tới R'_k . Vì Floyd rất quen thuộc trong OI, phần này không trình bày chi tiết. Độ phức tạp thời gian của Floyd là $O(n^3)$.

Trong đồ thị thưa, Floyd rất kém hiệu quả. Thực ra ta có thể trực tiếp liệt kê đỉnh cuối y , rồi tìm kiếm từ y trên đồ thị đảo để thu được mọi đỉnh x có thể tới y . Làm như vậy cho mọi y sẽ thu được bao đóng bắc cầu với độ phức tạp $O(nm)$.

Thuật toán trên có thể tối ưu bằng bitset, nhưng trước đó cần biến đồ thị thành không chu trình. Dùng nội dung sẽ giới thiệu ở mục 3.2, ta có thể co mỗi thành phần liên thông mạnh thành một đỉnh; khi đó đồ thị gốc trở thành một đồ thị có hướng không chu trình G' .

Đặt $f(x, y)$ biểu thị x có tới được y hay không. Xem $f(x, *)$ như một vector 01 và duy trì bằng bitset. Theo thứ tự tô pô đảo của G' , liệt kê từng đỉnh u . Khi đó $f(u, *)$ được lấy bằng phép or theo bit của mọi $f(v, *)$ với v là hậu duệ trực tiếp của u ; tất nhiên còn thêm khả đạt tới chính nó, $f(u, u) = 1$. Phép toán bit trên vector 01 độ dài n bằng bitset có độ phức tạp $O(n/w)$, nên tổng độ phức tạp là $O(nm/w)$.

Ví dụ về bài toán khả đạt động. Nói chung, bao đóng bắc cầu động là một bài toán tương đối khó; phần tài liệu tham khảo của bài viết có liệt kê các nghiên cứu liên quan. Loại thường gặp hơn là bài toán khả đạt động giữa một số cặp đỉnh cho trước. Với loại bài này, ta thường tiếp tục dùng phương pháp bitset để tính bao đóng bắc cầu, đồng thời kết hợp các kỹ thuật cấu trúc dữ liệu như chia khối truy vấn (hoặc chia khối thao tác).

Ví dụ 3.1.1 (Range Reachability Query⁸). Cho đồ thị có hướng không chu trình $G = (V, E)$, các cạnh có chỉ số. Có q truy vấn, mỗi truy vấn cho $u, v \in V$ và l, r , hỏi nếu chỉ giữ lại các cạnh có chỉ số trong $[l, r]$, thì u có tới được v hay không.

Vì chỉ có q truy vấn, ta có thể mô phỏng phương pháp bitset ở trên, nhưng đổi chiều thứ hai của bitset từ đỉnh thành truy vấn. Cụ thể, đặt $f(x, i)$ là: khi chỉ giữ các cạnh có chỉ số trong $[l_i, r_i]$, đỉnh x có tới được v_i hay không, trong đó l_i, r_i, v_i là các tham số tương ứng của truy vấn thứ i . Khi đó đáp án truy vấn thứ i là $f(u_i, i)$.

Xét cạnh $x \rightarrow y$ có chỉ số c . Chuyển trạng thái tương ứng là: với mọi i thỏa $l_i \leq c \leq r_i$, đặt $f(x, i) \leftarrow f(x, i) \vee f(y, i)$. Theo ngôn ngữ bitset, nếu S_c là tập mọi i thỏa $l_i \leq c \leq r_i$, được biểu diễn bằng vector 01 độ dài q , thì

$$f(x) \leftarrow f(x) \vee (f(y) \wedge S_c).$$

Dùng quét đường thẳng có thể tính mọi S_c , rồi chuyển trạng thái theo thứ tự tô pô đảo. Độ phức tạp thời gian là $O(q(m+n)/w)$, nhưng độ phức tạp bộ nhớ cũng là $O(q(m+n)/w)$, hơi lớn.

Loại bài này có một cách tối ưu bộ nhớ kinh điển: chia khối theo chiều được lưu bằng bitset. Trong bài này, chia khối truy vấn, mỗi khối gồm B truy vấn, và chạy thuật toán trên riêng cho từng khối. Như vậy bộ nhớ được tối ưu thành $O(B(m+n)/w)$, còn tổng thời gian vẫn là $O(q(m+n)/w)$. Tất nhiên độ dài khối B không được quá nhỏ, nếu không hệ số $1/w$ trong thời gian sẽ mất tác dụng.

Ví dụ 3.1.2 (Dynamic Reachability⁹). Cho đồ thị có hướng $G = (V, E)$, mỗi cạnh có màu đen hoặc trắng; ban đầu mọi cạnh đều màu đen. Sau đó có q thao tác, mỗi thao tác đảo màu một cạnh, hoặc cho $u, v \in V$ và hỏi u có thể tới v chỉ bằng cạnh đen hay không.

Vì màu cạnh thay đổi, bao đóng bắc cầu tĩnh trước đó khó áp dụng. Ta xét chia khối truy vấn; trong cùng một khối có nhiều cạnh không đổi màu, thuận lợi cho việc tiếp tục dùng phương pháp bitset.

Gọi độ dài khối là B . Trong một khối, tổng số đỉnh xuất hiện trong mọi sửa đổi và truy vấn nhiều nhất là $2B$; gọi các đỉnh này là đỉnh then chốt, lần lượt là $u_1, \dots, u_k$. Trước hết cần tính ảnh hưởng của các cạnh đen không bao giờ đổi màu trong khối. Gọi $f(i, j)$ biểu thị chỉ xét các cạnh đen không bao giờ đổi màu trong khối, i có tới được j hay không. Chú ý ta thực ra chỉ cần thông tin khi i, j đều là đỉnh then chốt, nên chiều thứ hai của bitset chỉ có kích thước $k = O(B)$ (vì chiều thứ hai chỉ cần ghi $u_1, \dots, u_k$). Độ phức tạp tiền xử lý phần này là $O(B(m+n)/w)$.

Lúc này ta nhận được một đồ thị có hướng G' chỉ chứa k đỉnh then chốt, trong đó có cạnh từ u_i tới u_j khi và chỉ khi $f(u_i, u_j) = 1$. Như vậy quy mô bài toán đã thu nhỏ; mọi ảnh hưởng của các đỉnh và cạnh không then chốt đều được thể hiện trong G' .

Xét cách trả lời truy vấn. Giả sử cần hỏi khả đạt từ u_i tới u_j . Ngoài các cạnh của G' , còn phải xét các cạnh đen mới xuất hiện do sửa đổi trong khối; số cạnh như vậy là $O(B)$. Ta có thể trực tiếp tìm kiếm thô để tìm tập đỉnh then chốt mà u_i tới được. Lấy BFS làm ví dụ: mỗi lần lấy một phần tử khỏi hàng đợi, cần đưa những đỉnh mới có thể thăm từ đỉnh đó vào hàng đợi. Quá trình này rõ ràng có thể tối ưu bằng bitset: chỉ cần duy trì tập mọi đỉnh hiện ở trong hàng đợi hoặc đã ra khỏi hàng đợi, ta nhanh chóng tìm được các đỉnh nên đưa vào hàng đợi. Độ phức tạp là $O(B^2/w)$.

⁸Huấn luyện đa trường HDU năm 2022.

⁹ZJCPC 2022, <https://codeforces.com/gym/103687>.

Tổng độ phức tạp thời gian là $O(q(n+m)/w + qB^2/w)$. Giống bài trước, độ dài khối không thể quá nhỏ, nếu không hệ số $1/w$ của bitset mất tác dụng. Vì vậy có thể lấy $B = w/2 = 32$, khi đó một số nguyên 64 bit đủ mô tả vector 01 có nhiều nhất $2B$ chiều. Trong bài này, lấy B lớn hơn một chút có vẻ chạy nhanh hơn.

Ví dụ 3.1.3 (Đồ thị có hướng không chu trình¹⁰). Cho đồ thị có hướng không chu trình $G = (V, E)$, đỉnh có trọng số, ban đầu mọi trọng số đỉnh bằng 0. Có q thao tác, gồm ba loại:

1. cho u, x , gán trọng số của mọi đỉnh mà u tới được thành x ;
2. cho u, x , lấy min với x trên trọng số của mọi đỉnh mà u tới được;
3. cho u , hỏi trọng số của u .

Bài này thực ra không phải bài toán khả đạt động, nhưng cách xử lý có nét tương tự.

Trước hết tính bao đóng bắc cầu của G với độ phức tạp $O(nm/w)$.

Truy vấn trọng số của u có thể tách thành hai bài toán con: trước hết tìm thao tác gán cuối cùng trước đó ảnh hưởng tới u , sau đó lấy giá trị nhỏ nhất trong các thao tác lấy min ảnh hưởng tới u từ thao tác đó tới thời điểm hiện tại.

Xét bài toán thứ nhất. Chia khối truy vấn với độ dài B . Mỗi khi xử lý xong một khối thao tác, dùng $O(n+m)$ để cập nhật thao tác gán cuối cùng của mỗi đỉnh. Khi truy vấn, ta đã biết đáp án trước khi khối hiện tại bắt đầu; chỉ cần kiểm tra trong khối hiện tại có thao tác gán nào liên quan tới u hay không. Việc này có thể làm bằng cách liệt kê trực tiếp mọi thao tác gán trong khối hiện tại, kiểm tra đỉnh u_i của thao tác đó có tới được u hay không.

Bài toán thứ hai được xử lý gần như giống vậy. Với mỗi u và mỗi khối, duy trì giá trị nhỏ nhất trong mọi thao tác lấy min của khối đó ảnh hưởng tới u . Truy vấn có dạng truy vấn đoạn theo u : với các khối nguyên dùng kết quả tiền xử lý, còn các phần lẻ liệt kê thô từng thao tác.

Độ phức tạp thời gian là $O(nm/w + qB + q(n+m)/B)$. Lấy $B = O(\sqrt{n+m})$, độ phức tạp là $O(nm/w + q\sqrt{n+m})$. Chú ý ở đây ta gặp vấn đề bộ nhớ giống ví dụ 3.1.1, có thể tối ưu tương tự: với mỗi khối thao tác, chiều lưu bằng bitset chỉ giữ những đỉnh được dùng trong các sửa đổi hoặc truy vấn của khối.

3.3.2 Liên thông mạnh

Định nghĩa 3.1 (liên thông mạnh). Trong đồ thị có hướng G , nếu hai đỉnh u, v thỏa u tới được v và v tới được u , ta nói u và v liên thông mạnh. Nếu mọi cặp đỉnh trong G đều liên thông mạnh, ta nói G là đồ thị liên thông mạnh.

Vì quan hệ khả đạt có tính bắc cầu, quan hệ liên thông mạnh cũng có tính bắc cầu, do đó là một quan hệ tương đương. Trong đồ thị có hướng G , ta có thể phân hoạch mọi đỉnh theo quan hệ liên thông mạnh.

Định nghĩa 3.2 (thành phần liên thông mạnh). Các đỉnh của đồ thị có hướng G có thể được phân hoạch thành vài lớp tương đương theo quan hệ liên thông mạnh. Đồ thị con cảm sinh bởi mỗi lớp tương đương được gọi là một thành phần liên thông mạnh của G .

Cây DFS và thuật toán Tarjan. Trong đồ thị có hướng G , nếu đỉnh r tới được mọi đỉnh, ta có thể bắt đầu DFS từ r . Khi đó, tương tự đồ thị vô hướng, ta định nghĩa cây DFS của G .

¹⁰Nguồn bài không truy vết được.

Định nghĩa 3.3 (thứ tự DFS, cây DFS). Trong đồ thị có hướng $G = (V, E)$, bắt đầu tìm kiếm theo chiều sâu từ một đỉnh $r \in V$ tới được mọi đỉnh. Nếu đỉnh x là đỉnh thứ p được thăm, gọi p là thứ tự DFS của x , ký hiệu $\text{dfn}_x = p$. Nếu $x \neq r$, nối một cạnh giữa x và đỉnh y lần đầu thăm được x ; đồ thị tạo thành được gọi là một cây DFS gốc r .

Cây DFS là một cây hướng ra ngoài. Trên cây DFS của đồ thị có hướng có vài tính chất tương tự đồ thị vô hướng.

Tính chất 3.2.1. Thứ tự DFS của mọi đỉnh tạo thành một hoán vị của $1, \dots, n$. Với mọi đỉnh x , tập thứ tự DFS của tất cả đỉnh trong cây con gốc x trên cây DFS tạo thành một đoạn liên tiếp bắt đầu từ dfn_x .

Tính chất 3.2.2. Cạnh không thuộc cây hoặc là cạnh từ hậu duệ tới tổ tiên (cạnh ngược), hoặc là cạnh từ tổ tiên tới hậu duệ (cạnh xuôi), hoặc là cạnh có hai đầu không có quan hệ tổ tiên và đi từ đỉnh có thứ tự DFS lớn tới đỉnh có thứ tự DFS nhỏ (cạnh chéo).

Chứng minh đơn giản, nhưng tính chất 3.2.2 cho thấy cấu trúc cạnh không thuộc cây trong đồ thị có hướng phức tạp hơn đồ thị vô hướng.

Tiếp theo xét cách dùng cây DFS để tìm thành phần liên thông mạnh. Quá trình tương tự tìm thành phần song liên thông: trong quá trình DFS vẫn ghi low_x , là thứ tự DFS nhỏ nhất của đỉnh có thể tới được từ một đỉnh bất kỳ trong cây con của x bằng cách đi qua nhiều nhất một cạnh không thuộc cây. Khi tìm được đỉnh x thỏa $\text{low}_x = \text{dfn}_x$, ta thu được một thành phần liên thông mạnh có x là đỉnh nông nhất.

Nhưng cách làm trên chưa hoàn toàn đúng, vì tồn tại cạnh chéo. Nếu trong cây con của x có một cạnh chéo trở tới một thành phần liên thông mạnh đã được thống kê trước đó, thì cạnh chéo này thực tế là vô hiệu (vì không giúp x đi tới vị trí nông hơn trên cây DFS). Ta cần bỏ qua các cạnh chéo như vậy, nên cải tiến thuật toán như sau:

Trong quá trình DFS, duy trì một ngăn xếp gồm các đỉnh đã thăm nhưng chưa được phân vào bất kỳ thành phần liên thông mạnh nào. Vẫn tính low_x trong DFS, nhưng chỉ khi đầu cuối của cạnh không thuộc cây còn nằm trong ngăn xếp (tức chưa bị đưa vào thành phần liên thông mạnh nào) thì cạnh đó mới được tính vào low_x . Khi gặp đỉnh $\text{low}_x = \text{dfn}_x$, lấy phần trong cây con của x đang nằm trên đỉnh ngăn xếp (chắc chắn là một đoạn liên tiếp ở đỉnh ngăn xếp) ra, cùng với x tạo thành một thành phần liên thông mạnh.

Chú ý rằng nếu ta chọn tùy ý một điểm xuất phát thì không nhất thiết DFS tới mọi đỉnh. Điều này không ảnh hưởng: nếu còn đỉnh chưa thăm, chọn tùy ý một đỉnh chưa thăm để DFS tiếp, lặp lại cho tới khi mọi đỉnh được thăm.

Đó là thuật toán Tarjan tìm thành phần liên thông mạnh, độ phức tạp thời gian $O(n + m)$. Ta không chứng minh chi tiết tính đúng đắn, nhưng sau khi hiểu thuật toán Tarjan trên đồ thị vô hướng và các tính chất của cây DFS trong đồ thị có hướng, mọi thứ trở nên tự nhiên.

Sau khi tìm tất cả thành phần liên thông mạnh, xem mỗi thành phần như một đỉnh. Cạnh trong đồ thị gốc nếu nằm hoàn toàn trong một thành phần thì bỏ qua, ngược lại xem là cạnh nối hai thành phần. Đồ thị có hướng mới thu được là một đồ thị có hướng không chu trình; quá trình này gọi là co đỉnh.

Hình 4: Một đồ thị có hướng, các thành phần liên thông mạnh khác nhau của nó, và kết quả co đỉnh.

Nếu bản thân G đã liên thông mạnh, thì nó chỉ có một thành phần liên thông mạnh là chính nó. Theo định nghĩa, bắt đầu DFS từ bất kỳ đỉnh nào cũng thăm được mọi đỉnh. Hơn nữa, theo thuật toán Tarjan, trên cây DFS, trong cây con của mọi đỉnh x đều phải tồn tại một cạnh trở tới một đỉnh có thứ tự DFS nhỏ hơn x , dù đó là cạnh ngược hay cạnh chéo.

Ví dụ 3.2.1 (Indiana Jones and the Uniform Cave¹¹). Đây là một bài tương tác. Có một đồ thị có hướng liên thông mạnh G , mỗi đỉnh có bậc ra bằng k . Tại mỗi đỉnh có một viên đá chỉ vào một cạnh ra, đồng thời có một nhãn thuộc $\{0, 1, 2\}$. Ban đầu ở mỗi đỉnh, viên đá chỉ ngẫu nhiên vào một cạnh ra và nhãn bằng 0.

Bạn không biết cấu trúc của G , nhưng cần xuất phát từ một đỉnh nào đó và duyệt qua mọi cạnh của G . Mỗi khi đi theo một cạnh tới một đỉnh, bạn biết nhãn của đỉnh đó, rồi có thể chọn sửa nhãn và hướng chỉ của viên đá, cuối cùng chọn một cạnh ra để rời đi. Tuy nhiên nhãn sau khi sửa chỉ có thể là 1 hoặc 2. Mọi cạnh ra của một đỉnh có thể xem như phân bố đều trên một đường tròn, nên khi mô tả một cạnh ra, bạn chỉ có thể dùng dạng “đếm thuận chiều kim đồng hồ từ cạnh đang được viên đá chỉ tới, cạnh thứ i ”.

Bạn chỉ được phép đi qua $O(nm)$ cạnh; chú ý ở mọi thời điểm bạn đều biết mọi quyết định trước đó của mình.

Ta thử mô phỏng quá trình DFS. Gọi quá trình tìm kiếm cây con của x là $dfs(x)$; ta kỳ vọng khi $dfs(x)$ được phép kết thúc, mọi cạnh ra của x đều đã được thăm.

Trong ba nhãn, 0 biểu thị tự nhiên cho đỉnh chưa thăm; quy định 1 và 2 lần lượt biểu thị đỉnh tương ứng là/không là tổ tiên của x , trong đó x là đỉnh đang thực hiện quá trình dfs hiện tại.

Với quá trình $dfs(x)$, lần lượt liệt kê mọi cạnh ra $x \rightarrow y$ và xét nhãn của y :

- Nhãn của y là 0. Điều này tương ứng y là con của x trên cây DFS. Ta đệ quy thực hiện $dfs(y)$, rồi quay lui.
- Nhãn của y là 1. Điều này nói $x \rightarrow y$ là cạnh ngược. Sau khi đi qua cạnh này, ta muốn quay về x . Để không lạc đường, quy định với đỉnh có nhãn 1, viên đá của nó chỉ vào cạnh nằm trên đường đi từ đỉnh đó tới x trong cây DFS. Như vậy chỉ cần đi theo cạnh được viên đá chỉ là có thể về x . Vấn đề là làm sao biết mình đã về tới x ? Câu trả lời là tạm thời sửa nhãn của x thành 2, khi lần sau thăm một đỉnh nhãn 2 thì chắc chắn đã về tới x .
- Nhãn của y là 2. Điều này nói $x \rightarrow y$ là cạnh xuôi hoặc cạnh chéo. Dù thế nào, ta vẫn muốn quay về x , nhưng lần này không tiện như trường hợp trước. Ta cần đi ngược hướng cây DFS cho tới khi tới một đỉnh nhãn 1, rồi quay về x theo cách ở trên (cách nhận biết đã về tới x hơi khác nhưng cũng đơn giản, nên không nói thêm). Vấn đề là làm sao đi tới một đỉnh nhãn 1. Nhắc lại mảng low trong thuật toán Tarjan: nếu từ đỉnh hiện tại u ta liên tục đi tới low_u , thì do đồ thị liên thông mạnh, chắc chắn có thể quay về đỉnh gốc, tức chắc chắn đi qua đỉnh nhãn 1. Vì vậy với mỗi đỉnh nhãn 2 là u , định nghĩa hướng viên đá của nó là hướng tới low_u . Nếu cạnh tương ứng với low_u là $u \rightarrow low_u$, viên đá chỉ vào cạnh đó; nếu cạnh tương ứng là $v \rightarrow low_u$ với v là hậu duệ của u , viên đá chỉ về hướng cây con chứa v .

Vấn đề cuối cùng: sau khi thực hiện xong $dfs(x)$, ta cần quay lui tới cha của x . Điều này thực ra giống trường hợp cuối cùng vừa nói; chỉ cần trước hết đi tới low_x , rồi đi xuống một đoạn là được.

Khi cài đặt có nhiều chi tiết, nhưng khung thuật toán đại thể đã rõ. Ta cần $O(nm)$ bước để duyệt mọi cạnh.

Phân rẽ tai. Trước hết nhắc lại phân rẽ tai đã định nghĩa ở phần đồ thị vô hướng và mở rộng nó sang đồ thị có hướng.

Định nghĩa 3.4 (tai và phân rẽ tai). Trong đồ thị có hướng $G = (V, E)$ có đồ thị con $G' = (V', E')$. Nếu đường đi đơn hoặc chu trình đơn $P : x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_k$ thỏa $x_1, x_k \in V'$ và $x_2, \dots, x_{k-1} \notin V'$, thì P được gọi là một tai của G đối với G' .

¹¹NEERC 2016, <https://codeforces.com/gym/101190>.

Nếu một dãy đồ thị con $(G_0, G_1, \dots, G_k)$ của G thỏa:

1. G_0 là một chu trình đơn (có thể chỉ có một đỉnh), $G_k = G$;
2. G_{i-1} là đồ thị con của G_i ;
3. đặt $G_i = (V_i, E_i)$, khi đó $E_i \setminus E_{i-1}$ tạo thành một tai của G_{i-1} ,

thì $(G_0, G_1, \dots, G_k)$ được gọi là một phân rã tai của G .

Tương tự đồ thị vô hướng, ta có kết luận sau.

Định lý 3.1. Đồ thị có hướng G có phân rã tai khi và chỉ khi G liên thông mạnh.

Chứng minh. Trước hết chứng minh tính cần. Nếu G có phân rã tai $(G_0, \dots, G_k)$, chu trình đơn G_0 là liên thông mạnh. Nếu G_i liên thông mạnh, giả sử tai được thêm vào G_{i+1} là $x \rightarrow y$. Từ tính liên thông mạnh của G_i , y tới được x , nên tồn tại một chu trình chứa tai; do đó mọi đỉnh trên tai liên thông mạnh với x, y , và G_{i+1} cũng liên thông mạnh. Quy nạp suy ra $G = G_k$ liên thông mạnh.

Tiếp theo chứng minh tính đủ. Nếu $G = (V, E)$ liên thông mạnh, ta đưa ra cách dựng một phân rã tai. Trước hết dựng một cây DFS gốc 1 của G , và tính mảng low theo thuật toán Tarjan. Tính liên thông mạnh của G cho biết khi $x \neq 1$ thì $\text{low}_x < \text{dfn}_x$. Sau đó xây $(G_0, \dots, G_k)$ bằng quá trình sau.

Cho G_0 là đồ thị chỉ chứa đỉnh 1. Sau đó liệt kê các đỉnh theo thứ tự DFS tăng dần. Giả sử đỉnh hiện tại là x có thứ tự DFS p ; khi đó mọi đỉnh có thứ tự DFS $1, \dots, p-1$ đã nằm trong G_i hiện tại. Nếu x đã ở trong G_i thì bỏ qua. Ngược lại, theo $\text{low}_x < \text{dfn}_x$, tồn tại một hậu duệ y của x và một cạnh $y \rightarrow z$ sao cho z đã nằm trong G_i . Khi đó lấy đường đi $f_x \rightarrow x \rightarrow y \rightarrow z$ làm một tai thêm vào G_i , trong đó f_x là cha của x ; thu được G_{i+1} . Sau khi liệt kê mọi đỉnh, mọi đỉnh đều đã nằm trong G_i hiện tại; lúc này thêm từng cạnh chưa được thêm vào như một tai riêng lẻ. ■

Ví dụ 3.2.2 (Economic One-way Roads¹²). Cho một đồ thị vô hướng G . Cần gán hướng cho mỗi cạnh; mỗi cạnh có chi phí riêng cho từng hướng. Hãy định hướng sao cho đồ thị có hướng thu được liên thông mạnh và tổng chi phí theo hướng được chọn nhỏ nhất.

Để đối chiếu, ví dụ này thực ra là phiên bản liên thông mạnh của ví dụ 2.2.4.

Gọi $f(S)$ là đáp án ứng với đồ thị con cảm sinh bởi S . Mỗi lần chuyển trạng thái bằng cách thêm một tai vào S . Nếu trực tiếp liệt kê tập đỉnh của tai, độ phức tạp là $O(3^n \text{Poly}(n))$; nếu ngay trong chuyển trạng thái thêm từng đỉnh theo thứ tự trên tai, có thể đạt $O(2^n \text{Poly}(n))$. Vì quá trình cụ thể gần như giống ví dụ 2.2.4, không nhắc lại chi tiết.

Định lý 3.2. Đồ thị vô hướng G có thể được định hướng thành đồ thị liên thông mạnh khi và chỉ khi G song liên thông cạnh.

Chứng minh. Tính cần: nếu G không song liên thông cạnh, thì G không liên thông hoặc có một cạnh cắt. Đồ thị không liên thông hiển nhiên không thể định hướng thành liên thông mạnh; nếu G có cạnh cắt (x, y) , dù định hướng cạnh này thế nào thì hoặc x không tới được y , hoặc y không tới được x , nên không thể có định hướng liên thông mạnh.

Tính đủ: nếu G song liên thông cạnh, nó có phân rã tai. Định hướng G_0 thành chu trình có hướng, rồi định hướng mỗi tai sau đó thành đường đi có hướng hoặc chu trình có hướng. Ta thu được một phân rã tai có hướng; theo định lý 3.1, đồ thị có hướng tương ứng là liên thông mạnh. ■

Thật ra việc dựng định hướng không cần phải tìm phân rã tai phức tạp như vậy. Ta chỉ cần dựng cây DFS của đồ thị song liên thông cạnh, định hướng mọi cạnh cây từ cha tới con, và định hướng mọi cạnh không thuộc cây từ hậu duệ tới tổ tiên. Điều này về cơ bản giống với cách dựng bằng phân rã tai ở trên.

¹²XXI Open Cup, Grand Prix of Korea, <https://codeforces.ml/gym/102759>.

Ví dụ 3.2.3 (trình quản lý App¹³). Một hỗn hợp đồ thị G (có thể đồng thời chứa cạnh có hướng và cạnh vô hướng) có thể được định hướng các cạnh vô hướng thành đồ thị liên thông mạnh khi và chỉ khi hai điều kiện sau cùng đúng:

1. nếu xem mọi cạnh có hướng như cạnh vô hướng, đồ thị song liên thông cạnh;
2. nếu tách mỗi cạnh vô hướng thành một cặp cạnh có hướng ngược nhau, đồ thị liên thông mạnh.

Chứng minh. Tính cần hiển nhiên. Dưới đây chứng minh tính đủ.

Nếu điều kiện 1 và 2 đều đúng, và không có cạnh vô hướng nào, ta không cần định hướng. Nếu còn cạnh vô hướng, lấy tùy ý một cạnh vô hướng (u, v) và chứng minh nó có một cách định hướng sao cho sau khi định hướng, điều kiện 1 và 2 vẫn đúng (thực ra chỉ cần xét điều kiện 2, vì điều kiện 1 không phụ thuộc hướng). Như vậy có thể chứng minh kết luận bằng quy nạp.

Gọi đồ thị hiện tại là G_0 , và đồ thị thu được từ G_0 sau khi xóa cạnh vô hướng (u, v) là G'_0 . Với một đỉnh bất kỳ x , do G_0 liên thông mạnh nên tồn tại một đường đi đơn $x \rightarrow u$. Nếu đường đi này không đi qua cạnh vô hướng (u, v) , thì trong G'_0 , x tới được u ; ngược lại trong G'_0 , x tới được v . Tương tự, trong G'_0 , hoặc u tới được x , hoặc v tới được x .

Trong G'_0 , nếu tồn tại x sao cho u tới được x và x tới được v , thì u tới được v . Tương tự, nếu tồn tại x sao cho v tới được x và x tới được u , thì v tới được u . Nếu không, với mỗi x đều là u, x tới được nhau hoặc v, x tới được nhau. Vì vậy toàn đồ thị có nhiều nhất hai thành phần liên thông mạnh. Giả sử u, v không tới được nhau, thì giữa hai thành phần liên thông mạnh này không có cạnh, nghĩa là đồ thị nền của G'_0 không liên thông, tức (u, v) là cạnh cắt trong G_0 , mâu thuẫn với điều kiện 1. Do đó ít nhất một trong hai quan hệ u tới được v , hoặc v tới được u , là đúng.

Nếu u tới được v , định hướng (u, v) thành $v \rightarrow u$; ngược lại định hướng thành $u \rightarrow v$. Điều này làm cho u, v tới được nhau, do đó toàn đồ thị vẫn liên thông mạnh, đúng như cần chứng minh. ■

Chu trình có hướng. Trong đồ thị vô hướng có không gian chu trình; mục này nghiên cứu cấu trúc chu trình của đồ thị có hướng.

Định nghĩa 3.5 (đồ thị phủ được bằng chu trình). Nếu trong đồ thị có hướng G , bậc vào của mỗi đỉnh bằng bậc ra của nó, ta nói G phủ được bằng chu trình.

Bổ đề 3.2.1. Tập cạnh của một đồ thị có hướng phủ được bằng chu trình có thể biểu diễn thành hợp rời của một số chu trình đơn.

Chứng minh xét bằng quy nạp. Khái niệm đồ thị phủ được bằng chu trình có thể so sánh với các phần tử của không gian chu trình trong đồ thị vô hướng.

Tương tự không gian chu trình, ta muốn dùng cây DFS để thu được một bộ “cơ sở” của các đồ thị con phủ được bằng chu trình; tuy nhiên nó không phải là cơ sở tuyến tính thật sự, nên không phải lúc nào cũng dùng được. Nói chung, ta xét trường hợp cạnh có trọng số (nếu không có quy định đặc biệt về trọng số, xem mỗi cạnh có trọng số 1), và nghiên cứu tính chất của tổng trọng số chu trình trong các cấu trúc như không gian xor hoặc các lớp dư modulo m .

Bổ đề 3.2.2. Trong đồ thị liên thông mạnh G , nếu tồn tại một đường đi $P : x \rightarrow y$ có tổng trọng số S , thì tồn tại một đường đi $y \rightarrow x$ có tổng trọng số S' thỏa $S' \equiv -S \pmod{m}$.

Chứng minh. Nếu $m = 1$ thì kết luận hiển nhiên. Ngược lại, lấy tùy ý một đường đi $Q : y \rightarrow x$ có tổng trọng số T . Xét đường đi ghép $Q + P + Q + P + \dots + Q$, gồm $m - 1$ bản sao của P và m bản

¹³UOJ Round 9, <https://uoj.ac/contest/18>.

sao của Q ; dấu cộng biểu thị ghép đường đi. Đây là một đường đi từ y tới x , và tổng trọng số của nó là $(m-1)S + mT \equiv -S \pmod{m}$. ■

Bổ đề 3.2.3. Trong đồ thị liên thông mạnh G , với mỗi cạnh $x \rightarrow y$ có trọng số z , thêm một cạnh mới $y \rightarrow x$ có trọng số $-z$. Ta thu được một đồ thị mới G' có trọng số cạnh phản đối xứng. Tập tổng trọng số của mọi đồ thị con phủ được bằng chu trình của G và G' là tương đương theo modulo m .

Chứng minh. Đây là hệ quả của bổ đề 3.2.2, vì mỗi chu trình của G' có thể tương ứng với một chu trình của G bằng bổ đề 3.2.2. ■

Vì vậy ta chỉ cần nghiên cứu đồ thị có trọng số phản đối xứng như vậy. Cây DFS của đồ thị này có thể xem như không còn một chiều: ta có thể đi hai chiều trên cây. Qua tính toán đơn giản, tổng trọng số của đường đi trên cây (không nhất thiết đơn) từ x tới y chắc chắn là $\text{dep}_y - \text{dep}_x$, trong đó dep_x là tổng trọng số trên đường đi từ gốc cây DFS tới x .

Bổ đề 3.2.4. Trên đồ thị G có trọng số phản đối xứng, gọi một chu trình đơn chỉ chứa một cạnh không thuộc cây (ở đây cạnh ngược chiều của cạnh cây không tính là cạnh không thuộc cây) là chu trình cơ bản. Khi đó tổng trọng số của mọi đồ thị con phủ được bằng chu trình của G bằng một tổ hợp tuyến tính hệ số nguyên của tổng trọng số một số chu trình cơ bản.

Chứng minh. Giả sử một chu trình lần lượt đi qua các cạnh không thuộc cây $e_1, \dots, e_k$, trong đó e_i có dạng $x_i \rightarrow y_i$ và trọng số z_i . Tổng trọng số chu trình là

$$\sum_{i=1}^k (z_i + \text{dep}_{y_{i+1}} - \text{dep}_{x_i}),$$

với $y_{k+1} = y_1$. Đổi chỗ các hạng chứa y_i trong tổng, ta được

$$\sum_{i=1}^k (z_i + \text{dep}_{y_i} - \text{dep}_{x_i}),$$

mà mỗi hạng đúng là tổng trọng số của một chu trình cơ bản. Kết hợp với tính chất đồ thị con phủ được bằng chu trình có thể tách thành một số chu trình, bổ đề được chứng minh. ■

Định lý 3.3. Trong đồ thị liên thông mạnh G , ước chung lớn nhất của tổng trọng số mọi đồ thị con phủ được bằng chu trình bằng ước chung lớn nhất của mọi $z + \text{dep}_y - \text{dep}_x$, trong đó (x, y, z) chạy qua bộ ba đỉnh đầu, đỉnh cuối và trọng số của mọi cạnh không thuộc cây, còn dep_x là độ dài đường đi trên cây từ gốc tới x .

Chứng minh. Gọi ước chung lớn nhất cần tìm là g . Theo bổ đề 3.2.3 và 3.2.4, ta biết trong modulo g , tập tổng trọng số các đồ thị con phủ được bằng chu trình của G tương đương với tập mọi $z + \text{dep}_y - \text{dep}_x$. Vì vậy nếu mọi phần tử của một trong hai tập đều là bội của g , thì tập còn lại cũng vậy. Điều này chứng minh hai cách tính ước chung lớn nhất là nhất quán. ■

Ở trên ta chỉ thảo luận tính chất theo các lớp dư modulo m . Tuy nhiên chỉ cần bổ đề 3.2.2 vẫn đúng, các tính chất trên có thể mở rộng sang một số cấu trúc khác, chẳng hạn không gian xor, hoặc tổng quát hơn là cấu trúc cộng không nhớ trong hệ cơ số k . Dù vậy, khi xét tính chất modulo m , chỉ cần nắm ước chung lớn nhất là cơ bản có được toàn bộ thông tin cấu trúc; còn trong các cấu trúc khác ở trên có thể cần thêm công cụ như cơ sở tuyến tính.

Ví dụ 3.2.4 (chu kỳ của đồ thị có hướng). Trong đồ thị có hướng $G = (V, E)$, nếu tồn tại một cặp số nguyên p, k sao cho với mọi cặp đỉnh x, y và mọi $t \geq k$, tồn tại một đường đi $x \rightarrow y$ gồm t cạnh khi và chỉ khi tồn tại một đường đi $x \rightarrow y$ gồm $t + p$ cạnh; đồng thời theo khóa thứ nhất là p và khóa thứ hai là k ,

cặp số nguyên nói trên là nhỏ nhất. Khi đó p, k lần lượt được gọi là một chu kỳ và chỉ số lũy thừa hội tụ của G .

Một cách nói tương đương: gọi A là ma trận kề của G , thì với mọi $t \geq k$ có $A^t = A^{t+p}$, trong đó phép nhân ma trận được định nghĩa bằng phép nhân là and theo bit, phép cộng là or theo bit.

Xét chu kỳ của đồ thị liên thông mạnh. Thực ra nó chính là ước chung lớn nhất g của mọi độ dài chu trình trong định lý 3.3, vì khi t đủ lớn, ta có thể đi tùy ý trong đồ thị liên thông mạnh. Theo định lý Bezout, ta có thể ghép ra bội bất kỳ của g bằng tổ hợp tuyến tính hệ số nguyên của độ dài các chu trình. Khi số bước đủ lớn, mọi hệ số đều có thể lấy là số nguyên dương.

Với đồ thị tổng quát, chu kỳ chắc chắn phải là bội của chu kỳ của từng thành phần liên thông mạnh; lấy bội chung nhỏ nhất của chu kỳ mọi thành phần liên thông mạnh là được. Điều này khá dễ hiểu, nên lược bỏ chứng minh nghiêm ngặt.

Với chỉ số lũy thừa hội tụ, không có công thức trực tiếp để tính.

Ví dụ 3.2.5 (Phoenix and Odometers¹⁴). Cho đồ thị có hướng có trọng số cạnh G . Có q truy vấn, mỗi truy vấn cho v, b, m , hỏi có tồn tại một chu trình (không nhất thiết đơn) đi qua v có tổng trọng số S thỏa $S \equiv b \pmod{m}$ hay không.

Đây là một ví dụ của cấu trúc lớp dư modulo m . Rõ ràng chỉ cần xét thành phần liên thông mạnh chứa v . Theo định lý 3.3, tính ước chung lớn nhất g của tổng trọng số mọi chu trình trong đó, rồi kiểm tra $\gcd(g, m) \mid b$ là đủ.

Ví dụ 3.2.6 (Thuật thụ số¹⁵). Cho một đồ thị vô hướng có trọng số G và số nguyên k . Có q truy vấn, mỗi truy vấn cho x, y, v , hỏi có tồn tại một đường đi $x \rightarrow y$ sao cho kết quả cộng không nhớ trong hệ cơ số k của mọi trọng số cạnh bằng v hay không.

Đây là một ví dụ của phép cộng không nhớ trong hệ cơ số k . Khác với trước đó, đây là đồ thị vô hướng, nên cần tách mỗi cạnh vô hướng thành một cặp cạnh có hướng ngược nhau. Vì vậy khi thực hiện thao tác tương tự định lý 3.3, ngoài cạnh không thuộc cây còn phải xét cạnh ngược chiều của cạnh cây; tức với cạnh cây có trọng số w , $2w$ cũng là một phần tử trong cơ sở. Sau đó dùng tất cả tổng trọng số chu trình cơ bản này để dựng một cơ sở tuyến tính hệ cơ số k phục vụ truy vấn.

Một điểm khác trong bài này là mục tiêu không phải chu trình mà là đường đi $x \rightarrow y$. Ta chỉ cần chọn tùy ý một đường đi $x \rightarrow y$, chẳng hạn đường đi trên cây, rồi cộng thêm một tập con các phần tử cơ sở là có thể thu được mọi đường đi từ x tới y . Vì vậy khi truy vấn, lấy tổng trọng số đường đi cây $x \rightarrow y$ làm giá trị ban đầu, rồi kiểm tra trong cơ sở tuyến tính có tồn tại một số phần tử sao cho cộng với giá trị ban đầu thành v hay không.

3.3.3 Đồ thị giải đấu

Đồ thị giải đấu là đồ thị có hướng đơn sao cho với mỗi cặp đỉnh khác nhau u, v , đúng một trong hai cạnh $u \rightarrow v$ và $v \rightarrow u$ tồn tại. Bây giờ ta thảo luận ngắn gọn vài kết luận liên quan tới tính liên thông trong đồ thị giải đấu.

Đường Hamilton.

Định nghĩa 3.6 (đường Hamilton, chu trình Hamilton). Trong đồ thị G , đường đi đi qua mọi đỉnh đúng một lần gọi là đường Hamilton; chu trình đi qua mọi đỉnh đúng một lần (đỉnh đầu và đỉnh cuối tính

¹⁴CF 1515 G

¹⁵CTT 2020; phát biểu bài ở đây không hoàn toàn giống bản gốc.

là một lần) gọi là chu trình Hamilton.

Định lý 3.4. Mọi đồ thị giải đấu đều có đường Hamilton.

Chứng minh. Quy nạp theo số đỉnh. Nếu đồ thị giải đấu $n - 1$ đỉnh có đường Hamilton, thì trong đồ thị giải đấu n đỉnh, giả sử không mất tính tổng quát rằng đường Hamilton của đồ thị con cảm sinh bởi $n - 1$ đỉnh đầu là $1 \rightarrow 2 \rightarrow \dots \rightarrow n - 1$. Khi đó:

1. nếu có cạnh $n \rightarrow 1$, nối n vào đầu đường Hamilton;
2. nếu có cạnh $n - 1 \rightarrow n$, nối n vào cuối đường Hamilton;
3. nếu có cạnh $1 \rightarrow n$ và $n \rightarrow n - 1$, thì chắc chắn tồn tại $x \in [1, n - 2]$ sao cho $x \rightarrow n$ và $n \rightarrow x + 1$ (có thể dùng nhị phân để tìm x như vậy), rồi chèn n vào giữa x và $x + 1$.

Vậy đồ thị giải đấu n đỉnh cũng có đường Hamilton. Theo quy nạp, kết luận đúng. ■

Định lý 3.5. Đồ thị giải đấu liên thông mạnh có chu trình Hamilton.

Chứng minh. Theo định lý 3.4, trước hết dựng một đường Hamilton, giả sử là $1 \rightarrow 2 \rightarrow \dots \rightarrow n$. Tìm t lớn nhất sao cho có cạnh $t \rightarrow 1$. Khi đó với các đỉnh x sau t , đều có cạnh $1 \rightarrow x$, tức $1 \rightarrow t + 1, \dots, 1 \rightarrow n$. Hiện ta có một chu trình $1 \rightarrow 2 \rightarrow \dots \rightarrow t \rightarrow 1$; tiếp theo cố gắng thêm các đỉnh $t + 1, \dots, n$ vào chu trình.

Mô tả lại bài toán: có một chu trình chứa 1, và từ 1 có cạnh tới mọi đỉnh chưa nằm trên chu trình. Ta muốn chứng minh tồn tại một đỉnh hiện chưa nằm trên chu trình có thể được thêm vào để tạo chu trình lớn hơn. Thật vậy, do đồ thị liên thông mạnh, chắc chắn tồn tại một đỉnh x chưa nằm trên chu trình và một đỉnh y trên chu trình sao cho có cạnh $x \rightarrow y$; nếu không, các đỉnh ngoài chu trình không tới được các đỉnh trên chu trình. Kết hợp với $1 \rightarrow x$, ta tìm được một vị trí thích hợp z trên chu trình (nằm giữa 1 và y) sao cho $z \rightarrow x \rightarrow \text{next}_z$. Chèn x vào giữa z và next_z , trong đó next_z là đỉnh kế tiếp của z trên chu trình. ■

Thành phần liên thông mạnh. Sau khi co thành phần liên thông mạnh, đồ thị tổng quát trở thành đồ thị có hướng không chu trình. Với đồ thị giải đấu, ta có kết quả tốt hơn.

Bổ đề 3.3.1. Sau khi co thành phần liên thông mạnh, đồ thị giải đấu tạo thành một dây chuyền.

Chứng minh. Với hai đỉnh x, y thuộc hai thành phần liên thông mạnh khác nhau, do chắc chắn tồn tại $x \rightarrow y$ hoặc $y \rightarrow x$, nên x tới được y hoặc y tới được x . Vì vậy quan hệ khả đạt giữa các thành phần liên thông mạnh tạo thành một thứ tự toàn phần, tức sau khi co đỉnh sẽ tạo thành một dây chuyền. ■

Trong đồ thị giải đấu, để tìm thành phần liên thông mạnh không cần dùng thuật toán Tarjan; có nhiều cách đơn giản hơn.

Trước hết có thể tìm một đường Hamilton của đồ thị giải đấu, giả sử là $1 \rightarrow 2 \rightarrow \dots \rightarrow n$. Khi đó mỗi thành phần liên thông mạnh chắc chắn là một đoạn liên tiếp. Với một cạnh $x \rightarrow y$, nếu $x < y$ thì nó không ảnh hưởng tới tính liên thông (vì trên dây chuyền đã có thể đi từ x tới y); nếu $x > y$, thì $y \rightarrow y + 1 \rightarrow \dots \rightarrow x$ hợp với cạnh này tạo thành một chu trình, tức mọi đỉnh trong $[y, x]$ thuộc cùng một thành phần liên thông mạnh. Hợp nhất tất cả các đoạn $[y, x]$ như vậy, ta nhận được một phân hoạch đoạn của $[1, n]$; các đoạn đó chính là mọi thành phần liên thông mạnh.

Còn một cách đơn giản và thực dụng hơn để tìm thành phần liên thông mạnh của đồ thị giải đấu, dựa vào định lý Landau dưới đây.

Định lý 3.6 (Landau). Giả sử trong đồ thị giải đấu G , mọi đỉnh được sắp theo bậc ra tăng dần thành $p_1, \dots, p_n$, và bậc ra của p_i là d_i . Khi đó với mọi $1 \leq k \leq n$:

$$\sum_{i=1}^k d_i \geq \binom{k}{2}.$$

Mở rộng: mỗi thành phần liên thông mạnh của G là một đoạn liên tiếp trong hoán vị p , và p_k là đầu phải của một thành phần liên thông mạnh khi và chỉ khi

$$\sum_{i=1}^k d_i = \binom{k}{2}.$$

Chứng minh. Chỉ xét đồ thị con cảm sinh bởi k đỉnh đầu, nó có $\binom{k}{2}$ cạnh; các cạnh này đã đóng góp $\binom{k}{2}$ vào $\sum_{i=1}^k d_i$, nên $\sum_{i=1}^k d_i \geq \binom{k}{2}$.

Nếu x, y thuộc hai thành phần liên thông mạnh khác nhau và x tới được y , thì với mọi cạnh ra $y \rightarrow z$ của y , dễ chứng minh chắc chắn tồn tại cạnh $x \rightarrow z$; ngoài ra x còn có thêm cạnh $x \rightarrow y$, nên bậc ra của x lớn hơn bậc ra của y . Điều này nói rằng mỗi thành phần liên thông mạnh đúng là một đoạn liên tiếp trong hoán vị p .

Nếu đẳng thức $\sum_{i=1}^k d_i = \binom{k}{2}$ xảy ra, thì không có cạnh nào từ $p_{k+1}, \dots, p_n$ tới $p_1, \dots, p_k$; do đó các đỉnh từ p_{k+1} trở đi không tới được các đỉnh trước p_k , và hai phía này không thể thuộc cùng một thành phần liên thông mạnh. Ngược lại, nếu $p_i, \dots, p_k$ tạo thành một thành phần liên thông mạnh, cũng suy ra các đỉnh sau p_{k+1} không tới được các đỉnh trước p_k , tức đẳng thức xảy ra. Điều này chứng minh phần mở rộng của định lý. ■

Trong định lý Landau, thay bậc ra bằng bậc vào thì kết luận tương ứng hiển nhiên cũng đúng. Dưới đây dùng mô tả theo bậc vào. Sắp mọi đỉnh theo bậc vào tăng dần; lấy các k thỏa $\sum_{i=1}^k d_i = \binom{k}{2}$ làm đầu phải, mỗi đoạn liên tiếp được chia ra chính là một thành phần liên thông mạnh, và các thành phần liên thông mạnh nằm bên trái có thể tới các thành phần nằm bên phải.

Ví dụ 3.3.1 (bài luyện tập lý thuyết đồ thị cơ sở¹⁶). Cho đồ thị giải đấu G . Với mỗi cạnh, hãy tính số thành phần liên thông mạnh sau khi lật hướng cạnh đó.

Cách 1: dùng đường Hamilton.

Nếu cạnh bị lật nối hai thành phần liên thông mạnh khác nhau, hiển nhiên sau khi lật sẽ hợp nhất thành một mọi thành phần liên thông mạnh nằm giữa hai thành phần này trên đường Hamilton; phần này rất dễ tính.

Nếu cạnh bị lật nằm trong cùng một thành phần liên thông mạnh, tìm một chu trình Hamilton của thành phần đó. Nếu cạnh bị lật không nằm trên chu trình Hamilton, rõ ràng tính liên thông mạnh không bị ảnh hưởng. Ngược lại, sau khi xóa cạnh đó trên chu trình Hamilton, còn lại một đường Hamilton. Như đã nói ở trên, với mọi cạnh $x \rightarrow y$ ngược hướng đường Hamilton, tác dụng của nó là hợp nhất đoạn $[y, x]$ thành một thành phần liên thông mạnh. Lúc này $[y, x]$ là đoạn trên chu trình; dùng một số kỹ thuật cấu trúc dữ liệu để duy trì mọi $[y, x]$, có thể truy vấn trong $O(1)$ tình hình thành phần liên thông mạnh sau khi xóa một cạnh trên chu trình, nhưng cài đặt khá rắc rối.

Cách 2: dùng định lý Landau.

Sắp mọi đỉnh theo bậc vào tăng dần thành $p_1, \dots, p_n$, bậc vào của p_i là d_i . Lật một cạnh $p_x \rightarrow p_y$ sẽ làm bậc vào của p_x tăng 1 và bậc vào của p_y giảm 1. Chỉ xét dãy d_i , phát hiện sửa đổi trên có thể biểu diễn bằng $O(1)$ phép sửa điểm trên d_i . Điều cần trả lời là số lượng k thỏa $\sum_{i=1}^k d_i = \binom{k}{2}$. Vì vậy chỉ cần duy trì trên mỗi đoạn giá trị nhỏ nhất của $\sum_{i=1}^k d_i - \binom{k}{2}$ và số lần đạt nhỏ nhất, là có thể trả lời đơn giản.

Cả hai cách đều có độ phức tạp thời gian $O(n^2)$, nhưng cách 2 gọn hơn.

¹⁶CTT 2020

3.4 Tổng kết

Bài viết đã thảo luận nhiều bài toán và thuật toán liên quan tới tính liên thông của đồ thị, đồng thời cung cấp một số cấu trúc và tư tưởng thường dùng để giải các bài toán này. Trong đồ thị vô hướng, ta lấy cây DFS làm trung tâm, tập trung thảo luận quan hệ song liên thông và một số tính chất, ứng dụng của tập cắt. Trong đồ thị có hướng, ta lấy quan hệ khả đạt và thành phần liên thông mạnh làm trung tâm, rồi thảo luận các tính chất và ứng dụng tương ứng.

Thực ra còn nhiều nội dung bài viết chưa đi sâu nhắc tới, chẳng hạn cắt nhỏ nhất trong đồ thị vô hướng, thành phần tam liên thông, quan hệ thống trị trong đồ thị có hướng, bao đóng bắc cầu động, v.v. Có hai lý do: một là giới hạn dung lượng, hai là các nội dung này đã được thảo luận trong các luận văn đội tuyển các năm trước. Tác giả liệt kê các luận văn tương ứng trong tài liệu tham khảo; độc giả quan tâm có thể đọc thêm.

3.5 Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
- Cảm ơn thầy Jin Jing của Trường Trung học phổ thông số 2 trực thuộc Đại học Sư phạm Hoa Đông đã quan tâm và hướng dẫn.
- Cảm ơn sự giúp đỡ của các bạn học đã trao đổi với tôi về các nội dung liên quan.
- Cảm ơn sự quan tâm và ủng hộ của cha mẹ.

3.6 Tài liệu tham khảo

- [1] Yu Haoxiang, “Bàn lại các thuật toán liên quan tới tính liên thông của đồ thị”, Tập luận văn đội tuyển IOI 2021.
- [2] Chang Ruinian, “Ứng dụng của đại số tuyến tính trong OI”, Tập luận văn đội tuyển IOI 2022.
- [3] Wikipedia, “Ear Decomposition”, https://en.wikipedia.org/wiki/Ear_decomposition.
- [4] Wikipedia, “Bipolar Orientation”, https://en.wikipedia.org/wiki/Bipolar_orientation.
- [5] zx2003, “Bàn sơ về định hướng lưỡng cực và ứng dụng”, <https://www.luogu.com.cn/blog/user19567/yi-dao-tu-lun-ti-ji-ji-yan-sheng>.
- [6] CCF, Niên giám Olympic Tin học Toàn quốc 2020.
- [7] Wang Wentao, “Bàn về một số thuật toán và ứng dụng của bài toán cắt nhỏ nhất trong đồ thị vô hướng”, Tập luận văn đội tuyển IOI 2016.
- [8] Sun Yaofeng, “Nghiên cứu bài toán bao đóng bắc cầu động”, Tập luận văn đội tuyển IOI 2017.
- [9] Chen Sunli, “Bàn về cây thống trị và ứng dụng”, Tập luận văn đội tuyển IOI 2020.

4 Bàn về xây dựng và sinh dữ liệu trong thi tin học

Trường Trung học phổ thông số 1 Weifang, tỉnh Sơn Đông
Liu Yiping

Tóm tắt

Bài viết này giới thiệu các phương pháp sinh ngẫu nhiên và xây dựng nhiều loại dữ liệu như dãy, cây, đồ thị, xâu ngoặc, đồng thời trình bày các lỗi hash thường gặp và cách xây dựng dữ liệu tương ứng.

4.1 Lời nói đầu

Trong thi tin học, ta thường cần kiểm tra tính đúng đắn của chương trình tại máy cục bộ.

Một phương pháp thường dùng là tự xây dựng dữ liệu kiểm thử. Với dữ liệu kích thước nhỏ, kết hợp với một chương trình khác có độ phức tạp cao hơn nhưng bảo đảm đúng, ta có thể kiểm tra tính đúng đắn của chương trình; với dữ liệu kích thước lớn, ta có thể kiểm tra hiệu suất của chương trình. Thí sinh thường gọi cách này là “đối phách”.

Trong đối phách, nếu cường độ dữ liệu sinh ra chưa đủ, có thể khó tìm ra lỗi của chương trình, hoặc để thuật toán có độ phức tạp cao vượt qua dữ liệu kích thước lớn.

Bài viết này sẽ giới thiệu vài cách sinh những loại dữ liệu thường gặp, cũng như cách sinh dữ liệu ngẫu nhiên hơn hoặc có tính nhắm mục tiêu hơn.

4.2 Dãy

4.2.1 Hoán vị

Thông thường có hai cách sinh hoán vị ngẫu nhiên. Chúng nhìn khá giống nhau, nhưng nguyên lý không giống nhau.

Cách thứ nhất như sau.

Thuật toán 1: Hoán vị ngẫu nhiên

Input: n

Output: một hoán vị ngẫu nhiên của 1 tới n

Đặt p là một dãy độ dài n , ban đầu $p_i = i$.

for $i = 1$ **to** n **do**

$j \leftarrow \text{random}(1, i)$

$\text{swap}(p_i, p_j)$

end for

return p

Ở đây $\text{random}(l, r)$ có tác dụng chọn đều một số nguyên trong $[l, r]$.

Bổ đề 1.1.1. Thuật toán 1 có thể sinh đều ngẫu nhiên một hoán vị của 1 tới n .

Chứng minh. Chú ý rằng thuật toán xáo trộn này trước hết thực hiện xáo trộn kích thước $n - 1$ trên $n - 1$ phần tử đầu của p , sau đó đặt $j = \text{random}(1, n)$ và hoán đổi p_n với p_j .

Xét quy nạp theo n .

Mệnh đề hiển nhiên đúng với $n = 1$; xét bước từ $n - 1$ suy ra n .

Sau khi hoán đổi ta có $p_j = n$. Vì j được chọn đều từ 1 tới n , chỉ cần chứng minh

$$p_1, \dots, p_{j-1}, p_{j+1}, \dots, p_n$$

là một hoán vị ngẫu nhiên đều.

Mặt khác, $p_1, \dots, p_{n-1}$ trước khi hoán đổi hiển nhiên song ánh với $p_1, \dots, p_{j-1}, p_{j+1}, \dots, p_n$ sau khi hoán đổi, nên hoán vị thu được là ngẫu nhiên đều. ■

Cách thứ hai như sau.

Thuật toán 2: Hoán vị ngẫu nhiên

Input: n

Output: một hoán vị ngẫu nhiên của 1 tới n

Đặt p là một dãy độ dài n , ban đầu $p_i = i$.

for $i = 1$ **to** n **do**

$j \leftarrow \text{random}(i, n)$

$\text{swap}(p_i, p_j)$

end for

return p

Bổ đề 1.1.2. Thuật toán 2 có thể sinh đều ngẫu nhiên một hoán vị của 1 tới n .

Chứng minh. Thuật toán này có thể xem là trước hết hoán đổi p_1 đều thành một trong các giá trị từ 1 tới n , rồi thực hiện thuật toán kích thước $n - 1$ trên p_2 tới p_n .

Xét quy nạp theo n , tính đúng đắn là hiển nhiên. ■

4.2.2 Dãy không có phần tử lặp

Xét bài toán sau: cần sinh đều ngẫu nhiên một dãy độ dài n , miền giá trị là các số nguyên từ 1 tới m ($m \geq n$), và các phần tử trong dãy không lặp.

Một phương pháp như sau.

Thuật toán 3: Dãy ngẫu nhiên không có phần tử lặp

Input: n

Output: một dãy ngẫu nhiên độ dài n không có phần tử lặp

Đặt a là một dãy độ dài n .

for $i = 1$ **to** n **do**

$a_i \leftarrow \text{random}(1, m)$

while $a_i \in \{a_1, a_2, \dots, a_{i-1}\}$ **do**

$a_i \leftarrow \text{random}(1, m)$

end while

end for

return p

Bổ đề 1.2.1. Thuật toán 3 có thể sinh đều ngẫu nhiên một dãy độ dài n , miền giá trị $[1, m] \cap \mathbb{Z}$, và các phần tử không lặp.

Chứng minh. Hiển nhiên, dãy được sinh ra chắc chắn hợp lệ, thuật toán có thể sinh mọi dãy hợp lệ, và xác suất sinh mọi dãy hợp lệ đều bằng nhau, đều là $1/m^n$. ■

Nhược điểm của phương pháp này là khi m tương đối nhỏ thì quá trình sinh sẽ khá chậm.

Ví dụ, khi $m = n$, số lần gọi random kỳ vọng có thể tính là

$$\sum_{i=1}^n n/i = \Theta(n \log n).$$

Việc kiểm tra $a_i \in \{a_1, a_2, \dots, a_{i-1}\}$ có thể làm bằng bảng hash trong $\Theta(1)$, nên tổng độ phức tạp là $\Theta(n \log n)$.

Nhưng chú ý rằng khi m tương đối nhỏ, có thể dùng một thuật toán khác: sinh một hoán vị ngẫu nhiên của 1 tới m , rồi lấy n phần tử đầu.

Khi $m \leq 2n$, dùng thuật toán thứ hai, độ phức tạp của phần này là $\Theta(n)$.

Khi $m > 2n$, dùng thuật toán thứ nhất; với mỗi i , xác suất mỗi lần chọn được một giá trị chưa xuất hiện là $(m - i + 1)/m \geq 1/2$, nên độ phức tạp kỳ vọng là $\Theta(n)$.

Một phương pháp khác là dùng thuật toán 2: ta chỉ cần cho vòng lặp i chạy tới n , dùng bảng hash lưu các vị trí đã bị thao tác và giá trị của chúng; độ phức tạp có thể đạt $\Theta(n)$.

4.2.3 Dãy có tổng cố định

Xét bài toán sau: cần sinh đều ngẫu nhiên một dãy số nguyên dương độ dài n , $a_1, a_2, \dots, a_n$, yêu cầu

$$\sum_{i=1}^n a_i = m.$$

Đặt $s_i = \sum_{j=1}^i a_j$, và chuyển sang sinh s .

Không khó thấy mọi s thỏa

$$1 \leq s_1 < s_2 < \dots < s_n = m$$

đều tương ứng một-một với a . Vì vậy, sinh đều ngẫu nhiên một a tương đương với sinh đều ngẫu nhiên một s như vậy.

Sinh s như vậy lại tương đương với: chọn đều ngẫu nhiên $n - 1$ số nguyên dương khác nhau trong $[1, m - 1]$. Điều này chuyển về bài toán sinh dãy không có phần tử lặp.

Với trường hợp a_i là số nguyên không âm, có thể cộng 1 vào mọi a_i để chuyển về trường hợp số nguyên dương.

Tổng quát hơn, nếu cận dưới của a_i là b_i , có thể trừ $b_i - 1$ khỏi a_i , từ đó chuyển về trường hợp cận dưới bằng 1.

4.3 Đồ thị

4.3.1 Cây

Cách 1: cha ngẫu nhiên. Một cách sinh cây ngẫu nhiên thường gặp trong thi tin học là: với mỗi $2 \leq i \leq n$, chọn đều ngẫu nhiên một đỉnh trong 1 tới $i - 1$ làm cha của i .

Với một cây có gốc, định nghĩa độ sâu của một đỉnh là số cạnh trên đường đi ngắn nhất từ nó tới gốc; định nghĩa chiều cao của cây là giá trị lớn nhất của độ sâu trên mọi đỉnh.

Bổ đề 2.1.1. Với mỗi $2 \leq i \leq n$, chọn đều ngẫu nhiên một đỉnh trong 1 tới $i - 1$ làm cha của i ; chiều cao kỳ vọng của cây sinh ra là $\Theta(\log n)$.

Bổ đề 2.1.2 (bất đẳng thức Jensen). Với hàm f lồi xuống trên đoạn $[L, R]$, với các số thực bất kỳ $x_1, x_2, \dots, x_n \in [L, R]$ và $a_1, a_2, \dots, a_n$ thỏa $\sum_{i=1}^n a_i = 1$ và $a_i > 0$, ta có

$$f\left(\sum_{i=1}^n a_i x_i\right) \leq \sum_{i=1}^n a_i f(x_i).$$

Chứng minh bổ đề 2.1.2 được lược bỏ; có thể tham khảo [1].

*Chứng minh bổ đề 2.1.1.*¹⁷ Giả sử chiều cao cây là h . Theo bất đẳng thức Jensen, có $2^{E[h]} \leq E[2^h]$. Xét cận trên của $E[2^h]$.

Gọi độ sâu của đỉnh i là d_i , khi đó

$$\begin{aligned} E[2^h] &= E\left[\max_{i=1}^n 2^{d_i}\right] \\ &\leq E\left[\sum_{i=1}^n 2^{d_i}\right] \\ &= \sum_{i=1}^n E[2^{d_i}]. \end{aligned}$$

Đặt $f_i = E[2^{d_i}]$, khi đó $f_1 = 1$, và với mọi $2 \leq i \leq n$ có

$$f_i = \frac{2}{i-1} \sum_{j=1}^{i-1} f_j.$$

Đặt $F_i = \sum_{j=1}^i f_j$, khi đó với mọi $2 \leq i \leq n$ có

$$F_i - F_{i-1} = \frac{2}{i-1} F_{i-1}.$$

Do đó

$$F_i = \frac{i+1}{i-1} F_{i-1} = \prod_{j=2}^i \frac{j+1}{j-1} = \frac{(i+1)i}{2}.$$

Vì vậy

$$2^{E[h]} \leq E[2^h] \leq F_n = \frac{(n+1)n}{2}.$$

Lấy log hai vế được $E[h] = O(\log n)$.

Theo chiều ngược lại,

$$E[h] \geq E\left[\frac{1}{n} \sum_{i=1}^n d_i\right] = \frac{1}{n} \sum_{i=1}^n E[d_i].$$

Đặt $g_i = E[d_i]$, khi đó $g_1 = 0$, và với $2 \leq i \leq n$ có

$$g_i = 1 + \frac{1}{i-1} \sum_{j=1}^{i-1} g_j.$$

Đặt $G_i = \sum_{j=1}^i g_j$, với $2 \leq i \leq n$ có

$$G_i - G_{i-1} = 1 + \frac{1}{i-1} G_{i-1}.$$

Do đó

$$G_i = 1 + \frac{i}{i-1} G_{i-1} = \sum_{j=2}^i \prod_{k=j}^{i-1} \frac{k+1}{k} = \sum_{j=2}^i \frac{i}{j} = \Theta(i \log i).$$

Vì vậy

$$E[h] \geq \frac{1}{n} \sum_{i=1}^n E[d_i] = \frac{1}{n} G_n = \Theta(\log n).$$

Nên $E[h] = \Omega(\log n)$.

Tổng hợp lại, $E[h] = \Theta(\log n)$. ■

¹⁷Một phần tham khảo từ [2].

Chiều cao kỳ vọng của cây là $\Theta(\log n)$; trong nhiều trường hợp, điều này không kiểm thử tốt hiệu suất thuật toán.

Ví dụ, một số thuật toán có độ phức tạp $\Theta(\sum_i \text{siz}_i) = \Theta(\sum_i \text{dep}_i)$, trong đó siz_i và dep_i lần lượt biểu thị kích thước cây con và độ sâu của đỉnh i . Trên loại dữ liệu này, độ phức tạp kỳ vọng là $\Theta(n \log n)$, trong khi độ phức tạp lớn nhất thực tế là $\Theta(n^2)$.

Nếu khi gộp heuristic không chọn con lớn nhất làm con nặng, hoặc trong một số tình huống sửa đổi trên đây chuyển xuất hiện lỗi, đều có thể làm độ phức tạp đạt tới $\Theta(\sum_i \text{dep}_i)$; dùng loại dữ liệu này thường không phát hiện được lỗi.

Cách 2: cha ngẫu nhiên trong một phạm vi nhất định. Một cách sửa đổi thường gặp của cách 1 là giới hạn phạm vi chọn cha ngẫu nhiên.

Ta có thể đặt một ngưỡng B , cha của đỉnh i được chọn ngẫu nhiên trong $[\max(1, i - B), i - 1]$.

Khi dùng phương pháp này, thường cần điều chỉnh kích thước B đúng lúc để sinh những loại dữ liệu khác nhau.

Thông thường B được đặt khá nhỏ; chẳng hạn với dữ liệu $n = 2 \times 10^5$ có thể đặt $B = 10$ hoặc 20, khi đó cây sinh ra khá phù hợp để kiểm thử gộp heuristic.

Cách 3: dãy Prufer. Với một cây vô hướng có nhãn, dãy Prufer của nó được sinh bằng cách sau.

Thuật toán 4: Dãy Prufer

Input: một cây vô hướng có nhãn T gồm n đỉnh

Output: dãy Prufer của T

Đặt a là một dãy độ dài $n - 2$.

for $i = 1$ **to** $n - 2$ **do**

$u \leftarrow$ lá có nhãn nhỏ nhất (đỉnh bậc 1)

$a_i \leftarrow$ nhãn của đỉnh kề với u

 Xóa u và các cạnh kề nó.

end for

return a

Có thể thấy, dãy Prufer của cây vô hướng có nhãn n đỉnh ($n \geq 2$) là một dãy độ dài $n - 2$, miền giá trị $[1, n] \cap \mathbb{Z}$.

Một cây tương ứng duy nhất với một dãy Prufer; vậy một dãy hợp lệ có tương ứng với một cây hay không?

Bổ đề 2.1.3. Dãy Prufer xây dựng một song ánh từ các cây n đỉnh tới các dãy độ dài $n - 2$, miền giá trị $[1, n] \cap \mathbb{Z}$.

Chứng minh. Chỉ cần chứng minh mỗi dãy Prufer đều tồn tại duy nhất một cây T tương ứng.

Xét quy nạp theo n . Trường hợp $n = 2$ đơn giản; xét dùng $n - 1$ suy ra n .

Giả sử dãy Prufer đã cho là $a_1, a_2, \dots, a_{n-2}$.

Không khó thấy số lần một nút xuất hiện trong dãy Prufer bằng bậc của nó trừ 1.

Từ đó có thể xác định toàn bộ lá trong cây; gọi lá có nhãn nhỏ nhất là u , khi đó đỉnh kề với u là a_1 .

Xét dãy Prufer $a_2, \dots, a_{n-2}$, với tập nhãn $\{1, \dots, n\} \setminus \{u\}$. Theo giả thiết quy nạp, tồn tại duy nhất một cây tương ứng với nó; gọi cây này là T' .

Thêm đỉnh u vào T' và nối cạnh với a_1 , nhận được cây T ; hiển nhiên đây là cây duy nhất tương ứng với a . ■

Chúng minh trên đồng thời cho một phương pháp dựng cây tương ứng từ dãy Prufer: dựng từ sau ra trước, mỗi lần thêm một lá.

Nếu muốn chọn đều ngẫu nhiên một cây vô hướng có nhãn trong tất cả các cây như vậy, chỉ cần chọn ngẫu nhiên dãy Prufer của nó. Cách sinh này còn có thể cố định bậc của từng đỉnh.

Cách 4: dây chuyền. Với đỉnh i ($2 \leq i \leq n$), chọn $i - 1$ làm cha của i là có thể sinh một dây chuyền.

Cách 5: cây nhị phân hoàn chỉnh. Với đỉnh i ($2 \leq i \leq n$), chọn $\lfloor i/2 \rfloor$ làm cha của i là có thể sinh một cây nhị phân hoàn chỉnh.

Với cây sinh theo cách này, nếu thực hiện phân rã nặng nhẹ, tổng kích thước các con nhẹ của mọi đỉnh là cấp $\Theta(n \log n)$.

Cách 6: kết hợp dây chuyền và cây nhị phân hoàn chỉnh. Ta có thể lấy hai ngưỡng B_1, B_2 : dùng B_1 đỉnh đầu để dựng cây nhị phân hoàn chỉnh, rồi cứ mỗi B_2 đỉnh phía sau nối thành một dây chuyền và gắn lên một trong B_1 đỉnh đầu.

Thông thường B_1 và B_2 đều được chọn ở cấp $\Theta(\sqrt{n})$.

Một cách chọn thường gặp là $B_1 = \lfloor 2\sqrt{n} \rfloor$, $B_2 = \lfloor \sqrt{n} \rfloor$. Trong tiểu mục 2.1.7 dưới đây ta cũng chọn như vậy.

Dữ liệu sinh theo cách này vừa có dây chuyền tương đối dài, vừa có tổng kích thước các con nhẹ của mọi đỉnh là $\Theta(n \log n)$.

Đối chiếu. Tiếp theo đối chiếu sáu phương pháp sinh.

Trước hết sinh một cây kích thước n , quy định đỉnh 1 là gốc. Sau đó xáo ngẫu nhiên các cạnh kề của từng đỉnh và thực hiện DFS. Cuối cùng thực hiện Q truy vấn; mỗi lần chọn ngẫu nhiên hai đỉnh u, v và truy vấn thông tin trên đường đi từ u tới v .

Quá trình trên được thực hiện T lần, sau đó đo các đại lượng dưới đây và lấy trung bình qua T lần:

- S_{size} biểu thị tổng kích thước cây con của mọi đỉnh. Một phần thuật toán độ phức tạp cao phụ thuộc vào đại lượng này.
- S_{son} biểu thị tổng kích thước các cây con nhẹ của mọi đỉnh sau khi phân rã nặng nhẹ. Nó có thể dùng để đánh giá hiệu suất gộp heuristic.
- S_{random} biểu thị tổng kích thước các cây con nhẹ của mọi đỉnh sau khi mỗi đỉnh chọn ngẫu nhiên một con làm con nặng. Nó có thể dùng để đánh giá hiệu suất của phương pháp gộp heuristic sai.
- S_{len} biểu thị độ dài đường đi trung bình trong Q truy vấn. Một phần thuật toán độ phức tạp cao phụ thuộc vào đại lượng này. Nó cũng có thể dùng để đánh giá hiệu suất của cấu trúc dữ liệu trên dây chuyền.
- S_{jump} biểu thị số cạnh nhẹ trung bình trên đường đi truy vấn sau khi phân rã nặng nhẹ. Nó có thể dùng để đánh giá hiệu suất của phân rã nặng nhẹ.

Lấy $n = 10^5$, $Q = 10^5$, $T = 10^3$, kết quả đo được như sau (giữ ba chữ số thập phân):

	S_{size}	S_{son}	S_{random}	S_{len}	S_{jump}
Cách 1	1212656.229	400679.705	873426.103	20.198	7.720
Cách 2 ($B = 10$)	909341143.306	116601.017	300801584.156	6069.061	2.332
Cách 2 ($B = 20$)	476433950.830	157012.831	166402833.765	3192.626	3.139
Cách 2 ($B = 40$)	244176851.716	198905.253	87602621.886	1664.193	3.972
Cách 3	39602294.635	325800.562	14507184.812	394.728	6.245
Cách 4	5000050000.000	0.000	0.000	33333.811	0.000
Cách 5	1568946.000	715030.000	734861.455	27.184	13.525
Cách 6	16675299.000	396342.000	418815.824	328.686	7.168

Có thể thấy:

- Cách 1: ưu điểm là mã ngắn, S_{son} tương đối lớn. Nhược điểm là S_{random} , S_{len} , S_{jump} đều quá nhỏ, khó kiểm thử các vấn đề độ phức tạp như gộp heuristic và phân rã nặng nhẹ.
- Cách 2: S_{size} , S_{random} , S_{len} lớn, và có xu hướng tăng khi B giảm.
- Cách 3: trong các loại dữ liệu được kiểm thử, các đại lượng có kích thước tương đối trung gian.
- Cách 4: S_{size} , S_{len} rõ ràng lớn; dãy chuyển khá phù hợp để kiểm thử hiệu suất của cấu trúc dữ liệu trên dãy chuyển ở kích thước cực hạn.
- Cách 5: S_{son} , S_{jump} lớn, khá phù hợp để kiểm thử hiệu suất của gộp heuristic và phân rã nặng nhẹ.
- Cách 6: cường độ nằm giữa cách 4 và cách 5.

4.3.2 Đồ thị tùy ý

Sinh đồ thị vô hướng n đỉnh m cạnh, không có cạnh bội và khuyên, chính là chọn m cạnh trong $\binom{n}{2}$ cạnh; có thể dùng kỹ thuật về dãy ở trên.

Nếu muốn sinh đồ thị liên thông, có thể liên tục sinh cho tới khi đồ thị liên thông.

4.3.3 Xương rỗng

Xương rỗng là đồ thị mà mỗi cạnh nằm trong nhiều nhất một chu trình đơn. Trong bài viết này, xương rỗng được phép có cạnh bội nhưng không được có khuyên.

Hiểu trực quan, xương rỗng là nhiều chu trình ghép lại thành hình dạng một cái cây. Cây cũng là một loại xương rỗng, nên xương rỗng thường được xem là mở rộng của cây.

Dưới đây giới thiệu ba cách sinh xương rỗng ngẫu nhiên.

Cách 1. Xét một đồ thị liên thông và thực hiện biến đổi sau trên nó. Dựng một đồ thị mới chứa toàn bộ đỉnh của đồ thị gốc, gọi là đỉnh tròn; với mỗi thành phần song liên thông đỉnh của đồ thị gốc, tạo thêm một đỉnh mới, gọi là đỉnh vuông (đặc biệt, ta xem một thành phần song liên thông đỉnh có thể chỉ chứa hai đỉnh và một cạnh). Nối mỗi đỉnh vuông với các đỉnh mà nó chứa trong đồ thị gốc; đồ thị thu được chắc chắn là một cây, gọi là cây tròn-vuông của đồ thị gốc.

Dễ thấy cây tròn-vuông thỏa các tính chất sau:

- Mỗi cạnh chắc chắn nối một đỉnh tròn với một đỉnh vuông. Nói cách khác, đỉnh tròn và đỉnh vuông tạo thành một tô màu đen-trắng của đồ thị này.
- Mọi đỉnh có bậc không quá 1 đều là đỉnh tròn.

Với xương rỗng, cây tròn-vuông của nó tương đương với việc tạo một đỉnh vuông cho mỗi chu trình.

Nếu muốn sinh ngẫu nhiên một xương rồng, trước hết ta có thể sinh ngẫu nhiên một cây, rồi tô màu đen-trắng cho nó, xem nó là cây tròn-vuông.

Nếu muốn cây tròn-vuông được sinh có n đỉnh, ta có thể sinh trước một cây $2n$ đỉnh, sau khi tô màu đen-trắng thì lấy màu có số lượng ít hơn làm đỉnh tròn; khi đó số đỉnh tròn chắc chắn không quá n .

Vì lá của cây tròn-vuông bắt buộc phải là đỉnh tròn, ta cần xóa mọi đỉnh vuông là lá.

Cách 2. Trước hết sinh ngẫu nhiên một cây, rồi thêm một số cạnh vào cây này để nó trở thành xương rồng.

DFS trên cây này; khi quay lui, với một xác suất nhất định nối tới một tổ tiên ngẫu nhiên, đồng thời đánh dấu các cạnh trên đường đi này, biểu thị rằng chúng đã nằm trên một chu trình. Vấn đề là xác định xác suất đó và tổ tiên được nối.

Ta có thể đặt một ngưỡng P . Khi quay lui ở đỉnh u , có thể xác định cạnh nối tới tổ tiên bằng cách sau.¹⁸

Thuật toán 5: Nối cạnh

```
 $v \leftarrow u$ 
while  $v$  không phải gốc và cạnh cha của  $v$  chưa bị đánh dấu do
     $x \leftarrow$  một số thực ngẫu nhiên trong  $[0, 1]$ 
    if  $x < P$  then
         $v \leftarrow$  cha của  $v$ 
    else
        break
    end if
end while
if  $u \neq v$  then
    Nối cạnh  $(u, v)$ 
    Đánh dấu đường đi từ  $u$  tới  $v$ 
end if
```

Ví dụ, lấy $P = 0.1$ thì các chu trình sinh ra sẽ tương đối nhỏ.

Cách 3. Vẫn xét cách trước hết sinh ngẫu nhiên một cây, rồi thêm cạnh.

Ta lấy một tham số K , sau đó chọn ngẫu nhiên K lần; mỗi lần chọn ngẫu nhiên hai đỉnh u, v và kiểm tra có thể nối một cạnh giữa u, v hay không; nếu có thì nối.

Cách kiểm tra tương tự cách 2: đánh dấu trên mỗi cạnh cây để biểu thị cạnh đó đã nằm trên một chu trình hay chưa. Khi kiểm tra, chỉ cần xét đường đi từ u tới v có cạnh đã đánh dấu hay không.

Nếu muốn bảo đảm độ phức tạp thời gian khi sinh dữ liệu quy mô lớn, cần dùng cấu trúc dữ liệu để duy trì đánh dấu. Nhưng trên thực tế, đi dọc đường đi từng bước và thoát khi gặp đánh dấu cũng có hiệu suất khá cao.

Đối chiếu. Ta kiểm thử T lần; mỗi lần chọn ngẫu nhiên một xương rồng, sau đó đo các đại lượng dưới đây và lấy trung bình qua T lần:

- N biểu thị số nút.
- C biểu thị số chu trình.

¹⁸Tham khảo từ [3].

- L biểu thị độ dài chu trình trung bình.
- σ biểu thị phương sai độ dài chu trình.
- M biểu thị độ dài chu trình lớn nhất.

Lấy $n = 10^5$, $T = 10^3$, phương pháp sinh cây là dãy Prufer ngẫu nhiên đều (tức cách 3 trong 2.1), kết quả đo như sau (giữ ba chữ số thập phân):

	N	C	L	σ	M
Cách 1	99909.960	44791.272	2.819	0.735	8.761
Cách 2 ($P = 0.1$)	100000.000	9889.691	2.110	0.122	5.728
Cách 2 ($P = 0.2$)	100000.000	19072.550	2.243	0.301	7.814
Cách 2 ($P = 0.4$)	100000.000	32611.841	2.567	0.853	11.825
Cách 3 ($K = 10^5$)	100000.000	221.323	35.973	2280.336	479.667

Có thể thấy:

- Cách 1: có thể dùng bậc của từng đỉnh để điều khiển phân bố độ dài chu trình, nhưng sẽ làm N giảm nhẹ. Khi dùng dãy Prufer ngẫu nhiên đều, chu trình khá ngắn, số chu trình nhiều, phương sai nhỏ.
- Cách 2: có thể điều chỉnh độ dài chu trình thông qua P ; khi P tăng, C, L, σ, M đều có xu hướng tăng. Nhìn chung chu trình khá ngắn, số chu trình nhiều, phương sai nhỏ.
- Cách 3: đặc điểm là số chu trình ít, độ dài chu trình lớn, phương sai lớn.

4.4 Xâu ngoặc

Xâu ngoặc là chuỗi được tạo bởi hai ký tự (và).

Với một xâu ngoặc $s = s_1 s_2 \dots s_n$, định nghĩa dãy đường đi $a_0, a_1, a_2, \dots, a_n$ của nó như sau:

$$a_i = \begin{cases} 0 & i = 0, \\ a_{i-1} + 1 & i > 0, s_i = (, \\ a_{i-1} - 1 & i > 0, s_i =). \end{cases}$$

Định nghĩa s là cân bằng khi và chỉ khi $a_n = 0$.

Định nghĩa độ khuyết defect(s) của s là số lượng các i thỏa $\min(a_i, a_{i+1}) < 0$ chia cho 2 (hiển nhiên số lượng đó luôn chẵn).

Định nghĩa s là hợp lệ khi và chỉ khi s cân bằng và độ khuyết bằng 0.

Định nghĩa $D_{n,k}$ là tập các xâu ngoặc cân bằng độ dài n , độ khuyết k .

Định nghĩa s^R biểu thị xâu sau khi đảo ngược s .

Với n chẵn, $D_{n,0}$ chính là tập các xâu ngoặc hợp lệ độ dài n ; theo số Catalan,

$$|D_{n,0}| = \frac{1}{n/2 + 1} \binom{n}{n/2}.$$

Với một xâu ngoặc cân bằng s , định nghĩa phép biến đổi $\Phi(s)$ như sau.¹⁹

- Nếu s rỗng, thì $\Phi(s)$ là xâu rỗng.

¹⁹Tham khảo từ [4].

- Nếu s không rỗng, chắc chắn có thể phân tích nó thành $s = lr$, trong đó l là tiền tố cân bằng không rỗng ngắn nhất của s ; dễ thấy r cũng là một xâu cân bằng.

Có thể thấy $|l|$ chính là i nhỏ nhất thỏa $i > 0, a_i = 0$.

Sau đó xét tính hợp lệ của l :

- Nếu l hợp lệ, thì $\Phi(s) = l\Phi(r)$.
- Nếu không, l chắc chắn có thể biểu diễn thành $l =)t($, và khi đó $\Phi(s) = (\Phi(r))t^R$.

Dễ chứng minh $\Phi(s)$ là một xâu ngoặc hợp lệ, tức $\Phi(s) \in D_{n,0}$.

Bổ đề 3.0.1. Với n chẵn và $0 \leq k \leq n/2$, Φ thiết lập một song ánh từ $D_{n,k}$ tới $D_{n,0}$.

Chứng minh. Trước hết chứng minh đơn ánh.

Xét quy nạp theo n ; mệnh đề hiển nhiên đúng với $n = 0$.

Xét hai xâu $s_1, s_2 \in D_{n,k}$ và $s_1 \neq s_2$; lần lượt phân tích chúng thành $s_1 = l_1 r_1, s_2 = l_2 r_2$, rồi phân loại.

- Trường hợp 1: l_1, l_2 đều hợp lệ.

Khi đó l_1, l_2 lần lượt là tiền tố cân bằng không rỗng ngắn nhất của $\Phi(s_1), \Phi(s_2)$. Nếu $l_1 \neq l_2$ thì mệnh đề đúng; nếu không, chắc chắn $r_1 \neq r_2$, theo giả thiết quy nạp có $\Phi(r_1) \neq \Phi(r_2)$, nên mệnh đề đúng.

- Trường hợp 2: l_1, l_2 đều không hợp lệ.

Khi đó $(\Phi(r_1))$ và $(\Phi(r_2))$ lần lượt là tiền tố cân bằng không rỗng ngắn nhất của $\Phi(s_1), \Phi(s_2)$; tiếp theo tương tự trường hợp 1, mệnh đề đúng.

- Trường hợp 3: l_1 hợp lệ, l_2 không hợp lệ. Đặt $l_2 =)t_2($.

Khi đó l_1 và $(\Phi(r_2))$ lần lượt là tiền tố cân bằng không rỗng ngắn nhất của $\Phi(s_1), \Phi(s_2)$.

Giả sử $\Phi(s_1) = \Phi(s_2)$, khi đó chắc chắn có $\Phi(r_1) = t_2^R$.

Vì l_1 hợp lệ nên $k = \text{defect}(s_1) = \text{defect}(r_1)$; do đó $2k \leq |r_1| = |\Phi(r_1)| = |t_2^R| < |l_2|$, tức $2k < |l_2|$.

Lại vì l_2 là tiền tố cân bằng không rỗng ngắn nhất của s_2 , chắc chắn có $k = \text{defect}(s_2) \geq \text{defect}(l_2)$, tức $2k \geq |l_2|$, mâu thuẫn.

Vì vậy $\Phi(s_1) \neq \Phi(s_2)$, mệnh đề đúng.

- Trường hợp 4: l_1 không hợp lệ, l_2 hợp lệ. Tương tự trường hợp 3.

Sau đó chứng minh toàn ánh.

Vì đã có đơn ánh, nên với mọi $0 \leq k \leq n/2$ có $|D_{n,k}| \leq |D_{n,0}|$. Ta chỉ cần chứng minh $|D_{n,k}| = |D_{n,0}|$.

Giả sử tồn tại $0 \leq k \leq n/2$ thỏa $|D_{n,k}| < |D_{n,0}|$, khi đó

$$\binom{n}{n/2} = \sum_{k=0}^{n/2} |D_{n,k}| < (n/2 + 1)|D_{n,0}| = \binom{n}{n/2},$$

mâu thuẫn.

Tổng hợp lại, với mọi $0 \leq k \leq n/2$, Φ thiết lập một song ánh từ $D_{n,k}$ tới $D_{n,0}$. ■

Có bổ đề 3.0.1, ta có thể chọn ngẫu nhiên một xâu ngoặc hợp lệ rất tiện lợi.

Chỉ cần chọn đều ngẫu nhiên một trong tất cả $\binom{n}{n/2}$ xâu cân bằng, rồi áp dụng Φ lên nó.

4.5 Hash

Hash là thuật toán rất thường dùng trong thi tin học; nó nén thông tin bằng một cách tương đối ngẫu nhiên để tăng tốc việc so sánh thông tin.

Trong tình huống thông thường, muốn làm hash sai thì phải xây dựng dữ liệu nhắm vào mã nguồn cụ thể.

Tuy nhiên có một số phương pháp hash không cần nhắm vào mã nguồn; có thể trực tiếp xây dựng dữ liệu làm chúng sai, còn dữ liệu ngẫu nhiên dùng khi đối phó thường không đủ để phát hiện những vấn đề này.

Dưới đây liệt kê vài thuật toán hash thường gặp, dễ bị xây dựng dữ liệu làm sai.

4.5.1 Nghịch lý sinh nhật

Trong một căn phòng có 23 người. Giả sử một năm luôn có 365 ngày và sinh nhật của mỗi người được chọn độc lập, đều trong 365 ngày này. Hỏi: xác suất tồn tại hai người có cùng sinh nhật là bao nhiêu?

Giả sử một năm có n ngày, trong phòng có m người. Khi đó xác suất sinh nhật của m người này đôi một khác nhau là

$$\prod_{i=1}^m \frac{n-i+1}{n}.$$

Thay $n = 365, m = 23$ vào, xác suất trên xấp xỉ 49%, tức xác suất tồn tại hai người có cùng sinh nhật lớn hơn 50%. Điều này rất trái trực giác, nên ta gọi hiện tượng này là nghịch lý sinh nhật.

Vậy m cần đạt tới bao nhiêu để xác suất tồn tại hai người cùng sinh nhật không nhỏ hơn 50%?

Có thể lập phương trình sau:

$$\prod_{i=1}^m \frac{n-i+1}{n} \leq \frac{1}{2}.$$

Dùng $1+x \leq e^x$ để ước lượng:

$$\prod_{i=1}^m e^{-\frac{i-1}{n}} \leq \frac{1}{2}.$$

Tức là

$$e^{-\frac{m(m-1)}{2n}} \leq \frac{1}{2}.$$

Suy ra $m \approx \sqrt{2n \ln 2}$.

Phương pháp hash thông thường là chọn mô-đun M , nén thông tin cần hash thành một số nguyên trong $[0, M-1]$.

Quá trình này có thể gần đúng xem là chọn ngẫu nhiên một số nguyên trong $[0, M-1]$.

Nếu bài toán cần xử lý có dạng “phán đoán trong n thông tin có trùng lặp hay không”, thì khi n xấp xỉ $\sqrt{2M \ln 2}$, xác suất phán thông tin không trùng thành có trùng sẽ đạt tới $\frac{1}{2}$.

Mô-đun hash đơn thông thường ở cấp 10^9 ; khi n đạt xấp xỉ 3×10^4 , xác suất xuất hiện va chạm khoảng 36%, một xác suất rất lớn.

Trong trường hợp này, nên chọn hai mô-đun nguyên tố cùng nhau M_1, M_2 ; khi đó miền giá trị ngẫu nhiên có thể xem là $M_1 M_2$, làm xác suất va chạm giảm mạnh.

4.5.2 Tràn tự nhiên

Khi làm hash xâu, thông thường ta chọn mô-đun M và cơ số B . Với xâu $s = s_1 s_2 \dots s_n$, giá trị hash của nó được tính là

$$\text{hash}(s) = \left(\sum_{i=1}^n B^{i-1} \text{ord}(s_i) \right) \bmod M.$$

Ở đây $\text{ord}(c)$ biểu thị vị trí của ký tự c trong bảng mã.

Số nguyên không dấu 64 bit trong C++ sẽ tự động lấy mô-đun 2^{64} khi tính toán; nếu chọn $M = 2^{64}$, có thể bớt một phép lấy mô-đun để tăng tốc. Phương pháp hash này gọi là tràn tự nhiên.

Nhưng phương pháp hash này có thể rất dễ bị xây dựng và chậm.

Nếu B là số chẵn, thì $B^{64} \bmod M = 0$, tức trong s nhiều nhất chỉ có 64 vị trí cuối là có hiệu lực, rất dễ xây dựng và chậm. Dưới đây xét trường hợp B là số lẻ.

Xét xâu $s = s_1 s_2 \dots s_{2^k}$ thỏa

$$s_i = \begin{cases} A & \text{popcount}(i-1) \text{ là số chẵn,} \\ B & \text{popcount}(i-1) \text{ là số lẻ.} \end{cases}$$

Trong đó $\text{popcount}(x)$ biểu thị số bit 1 trong biểu diễn nhị phân của x .

Với xâu t , đặt t^F là kết quả thay mọi A trong t bằng B và đồng thời thay mọi B bằng A .

Xét dãy xâu $t_0, t_1, \dots, t_k$ thỏa $t_0 = A$, $t_i = t_{i-1} + t_{i-1}^F$, khi đó $s = t_k$.

Đặt $d = \text{ord}(A) - \text{ord}(B)$, dễ thu được

$$\text{hash}(s) - \text{hash}(s^F) \equiv \sum_{i=1}^{2^k} (-1)^{\text{popcount}(i-1)} B^{i-1} d \pmod{M}.$$

Ta cần chứng minh khi k đủ lớn, giá trị này luôn bằng 0 theo nghĩa mod M .

Đặt

$$f_i = \text{hash}(t_i) - \text{hash}(t_i^F).$$

Dễ thu được

$$f_i = f_{i-1} (1 - B^{2^{i-1}}).$$

Mà $f_0 = d$, nên

$$f_i = d \prod_{j=0}^{i-1} (1 - B^{2^j}).$$

Chú ý rằng

$$1 - B^{2^j} = (1 - B^{2^{j-1}})(1 + B^{2^{j-1}}).$$

Tức $1 - B^{2^{j-1}}$ nhân với một số chẵn. Nếu $2^p \mid (1 - B^{2^{j-1}})$ thì chắc chắn $2^{p+1} \mid (1 - B^{2^j})$.

Do đó f_i chắc chắn chia hết cho $2^1 2^2 \dots 2^i = 2^{\frac{i(i+1)}{2}}$.

Khi $k = 11$, có $2^{64} \mid f_k$, tức $\text{hash}(t_k) = \text{hash}(t_k^F)$.

4.5.3 Hash cây

Hash cây thường được dùng để phán đoán hai cây có gốc có đẳng cấu hay không.

Một phương pháp hash cây tương đối đúng là dùng thứ tự ngoặc và tư tưởng biểu diễn nhỏ nhất, để các cây đẳng cấu nhận được cùng một thứ tự ngoặc.

Cụ thể, giá trị hash của một cây con u có thể được tính đệ quy như sau:

1. Tính giá trị hash của mọi con của u .
2. Sắp xếp chúng theo khóa (giá trị hash, kích thước cây con).
3. Theo thứ tự đã sắp xếp, nối thứ tự ngoặc của mọi con của u lại.
4. Thêm một cặp ngoặc ở ngoài cùng.
5. Dây thu được chính là thứ tự ngoặc của u , và giá trị hash của nó là giá trị hash của u .

Để chứng minh các cây đẳng cấu nhận được cùng giá trị hash.

Việc thêm kích thước cây con vào khóa sắp xếp ở bước thứ hai là để ngăn các cây con khác kích thước nhưng có giá trị hash bằng nhau do nghịch lý sinh nhật làm ảnh hưởng kết quả.

Giả sử số nút của cây là n ; vì cần sắp xếp, độ phức tạp của thuật toán trên là $\Theta(n \log n)$.

Còn có một thuật toán hash độ phức tạp $\Theta(n)$ được biết đến rộng rãi.

Gọi kích thước cây con của đỉnh u là size_u , tập con của nó là son_u .

Ta sinh một dãy $W_1, W_2, \dots, W_n$, thông thường dùng dãy số nguyên tố hoặc dãy ngẫu nhiên.

Sau đó giá trị hash của đỉnh u được tính như sau:

$$h_u = W_{\text{size}_u} \left(1 + \sum_{v \in \text{son}_u} h_v \right).$$

Xét đường đi từ đỉnh u tới gốc; viết lần lượt các giá trị size trên đường đi này, thu được dãy S_u .

Chú ý rằng giá trị hash của toàn bộ cây T chỉ liên quan tới đa tập không thứ tự

$$H(T) = \{S_1, S_2, \dots, S_n\};$$

hai cây có tập này bằng nhau chắc chắn nhận được giá trị hash bằng nhau.

Vấn đề của kiểu hash này là đôi khi chỉ dựa vào $H(T)$ thì không thể xác định duy nhất hình dạng của cây.

Ví dụ, bất kể chọn W như thế nào, hai cây dưới đây chắc chắn có giá trị hash bằng nhau:

$$T_1 : \{(1, 2), (1, 8), (2, 3), (2, 4), (2, 5), (5, 6), (5, 7), (8, 9), (8, 12), (9, 10), (10, 11), (12, 13)\},$$

$$T_2 : \{(1, 2), (1, 8), (2, 3), (2, 4), (4, 9), (9, 10), (10, 11), (8, 5), (8, 12), (5, 6), (5, 7), (12, 13)\}.$$

Thực ra, chỉ cần hoán đổi hai cây con có S bằng nhau là có thể giữ $H(T)$ không đổi. Trong hình gốc, hai cây con của 5 và 9 đã được hoán đổi; các tập cạnh ở trên mô tả đúng hai cây đó.

Bộ dữ liệu này có thể làm sai mọi thuật toán hash chỉ dựa trên $H(T)$.

4.6 Tổng kết

Các kiểu dữ liệu mà bài viết này giới thiệu là những kiểu khá thường gặp trong thi tin học.

Các kiểu dữ liệu như dãy, cây suy sinh đơn giản, nhưng nếu phương pháp không phù hợp thì dễ xây dựng ra dữ liệu có cường độ chưa đủ.

Các kiểu dữ liệu như xương rồng, xoắn ngoặc lại có độ khó nhất định khi sinh.

Còn nếu phương pháp hash không phù hợp, thí sinh có thể vẫn mất điểm dù đã vượt qua dữ liệu ngẫu nhiên.

Trong nhiều trường hợp hơn, thí sinh cần dựa vào tính chất của bài toán để xây dựng dữ liệu có tính nhắm mục tiêu hơn. Hy vọng bài viết này có thể gợi mở suy nghĩ cho thí sinh, giúp hiểu sâu hơn về phương pháp kiểm thử chương trình và xây dựng dữ liệu.

4.7 Lời cảm ơn

- Cảm ơn cha mẹ đã nuôi dưỡng, quan tâm và ủng hộ.
- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
- Cảm ơn hai huấn luyện viên đội tuyển quốc gia Yang Yaoliang và Peng Sijin đã hướng dẫn.
- Cảm ơn thầy Leng Xuenong đã bồi dưỡng và chỉ dạy.
- Cảm ơn các thầy cô và bạn học đã động viên, giúp đỡ.

4.8 Tài liệu tham khảo

- [1] Wikipedia, “Bất đẳng thức Jensen”, <https://zh.wikipedia.org/wiki/%E7%B0%A1%E6%A3%AE%E4%B8%8D%E7%AD%89%E5%BC%8F>.
- [2] “Nhật ký hoạt động 2020.11.21: chiều cao cây kỳ vọng của cây cha ngẫu nhiên”, https://blog.csdn.net/EI_Captain/article/details/109910307.
- [3] “Chơi với xương rồng”, <https://vfleaking.blog.uoj.ac/blog/89>.
- [4] M.D. Atkinson and J.-R. Sack, “Generating binary trees at random”, <http://www.cs.otago.ac.nz/staffpriv/mike/Papers/RandomGeneration/RandomBinaryTrees.pdf>.

5 Bàn về một lớp bài toán có treewidth bị chặn

Trường Thập Nhất Bắc Kinh
Wu Chang

Tóm tắt

Bài viết giới thiệu các khái niệm liên quan tới treewidth, cách phân rã cây của đồ thị có treewidth bị chặn, và một số thuật toán tốt cho các bài toán kinh điển trên lớp đồ thị này. Nội dung đi từ định nghĩa, tính chất cơ bản và các đồ thị đặc biệt, tới ý nghĩa thuật toán của treewidth bị chặn và phác thảo thuật toán tuyến tính của Bodlaender để tìm phân rã cây khi treewidth là hằng số.

5.1 Mở đầu

Treewidth là một khái niệm quan trọng trong khoa học máy tính. Độ phức tạp của nhiều bài toán trên đồ thị tăng rất nhanh theo treewidth, trong khi nhiều đồ thị phát sinh trong thực tế, chẳng hạn mạng đường, lại thường có treewidth nhỏ. Vì vậy, việc tìm phân rã cây có độ rộng nhỏ và giải các bài toán trên đồ thị có treewidth nhỏ đã trở thành một hướng nghiên cứu được quan tâm.

Trong thi tin học cũng từng xuất hiện các bài toán trên đồ thị đặc biệt có treewidth nhỏ, nhưng khái niệm và thuật toán liên quan chưa phổ biến. Bài viết trình bày các định nghĩa cần dùng, một số quy ước và tính chất cơ bản, phân tích treewidth của vài lớp đồ thị đặc biệt, nhấn mạnh quá trình thu gọn của đồ thị nối tiếp-song song tổng quát, rồi giới thiệu ý nghĩa của treewidth bị chặn và một số thuật toán cổ điển trên lớp đồ thị này.

5.2 Định nghĩa

Định nghĩa 2.1 (phân rã cây). Với đồ thị vô hướng $G = (V, E)$, nếu một cây $T = (I, F)$ và một họ tập đỉnh

$$\{X_i \subseteq V \mid i \in I\}$$

đồng thời thỏa ba điều kiện sau, thì (X, T) được gọi là một phân rã cây của G :

- $\bigcup_{i \in I} X_i = V$;
- với mọi cạnh $(u, v) \in E$, tồn tại $i \in I$ sao cho $u, v \in X_i$;
- với mọi $i, j, k \in I$, nếu j nằm trên đường đi giữa i và k trong T , thì $X_i \cap X_k \subseteq X_j$.

Các tập X_i thường được gọi là các *bag*.

Định nghĩa 2.2 (độ rộng). Độ rộng của phân rã cây (X, T) là $\max_{i \in I} |X_i| - 1$.

Định nghĩa 2.3 (treewidth). Treewidth của G , ký hiệu $\text{tw}(G)$, là độ rộng nhỏ nhất trong tất cả các phân rã cây của G .

Định nghĩa 2.4 (k -tree). Một k -tree bắt đầu từ đồ thị đầy đủ trên $k + 1$ đỉnh. Sau đó lặp lại thao tác thêm một đỉnh mới, sao cho đỉnh mới nối đúng với k đỉnh cũ và k đỉnh này đôi một kề nhau.

Định nghĩa 2.5 (partial k -tree). Một partial k -tree là đồ thị con bất kỳ của một k -tree.

5.3 Quy ước và tính chất cơ bản

Định lý 3.1. $\text{tw}(G) \leq k$ khi và chỉ khi G là một partial k -tree. Chứng minh của định lý này khá dài nên bài viết lược bỏ.

Nếu không nói thêm, các phần sau xét đồ thị G không rỗng, không có cạnh bội và không có khuyên. Với cây T có gốc, lấy tùy ý $r \in I$ làm gốc. Ký hiệu dep_i là độ sâu của nút i trong T , và $\text{subtree}(i)$ là tập nút trong cây con của i .

Với $u \in V$, đặt

$$S_u = \{i \in I \mid u \in X_i\}.$$

Ký hiệu $N_G(u) = \{v \mid (u, v) \in E\}$ và $d_G(u) = |N_G(u)|$. Gọi top_u là nút nông nhất trong S_u , và đặt $\text{lev}_u = \text{dep}_{\text{top}_u}$.

Bổ đề 3.0.1. Với mọi cạnh $(u, v) \in E$, giả sử không mất tổng quát rằng $\text{lev}_u \geq \text{lev}_v$. Khi đó $v \in X_{\text{top}_u}$.

Chứng minh. Vì $S_u \subseteq \text{subtree}(\text{top}_u)$ và $S_u \cap S_v \neq \emptyset$, nếu $\text{top}_u \notin S_v$ thì toàn bộ S_v cũng nằm trong cây con của top_u , mâu thuẫn với $\text{lev}_u \geq \text{lev}_v$.

Bổ đề 3.0.2. Nếu G là partial k -tree và $k < |V|$, thì

$$|E| \leq k|V| - \frac{k(k+1)}{2}.$$

Chứng minh. Lấy đỉnh u có lev lớn nhất trong V . Theo bổ đề trước, $d_G(u) \leq k$. Xóa u rồi quy nạp.

Thu gọn cạnh $(u, v) \in E$ nghĩa là xóa cạnh này, hợp nhất u, v thành một đỉnh mới và để đỉnh mới thừa hưởng các cạnh kề ban đầu.

Bổ đề 3.0.3. Nếu thu gọn cạnh của G được G' , thì $\text{tw}(G') \leq \text{tw}(G)$.

Chứng minh. Giả sử đỉnh mới là w . Với một phân rã cây (X, T) của G , nếu X_i chứa ít nhất một trong hai đỉnh u, v , thay chúng bằng w ; nếu không, giữ nguyên X_i . Họ bag thu được là một phân rã cây của G' và độ rộng không tăng.

5.4 Các đồ thị đặc biệt

5.4.1 Partial 1-tree

Định lý 4.1. Partial 1-tree chính là rừng.

Chứng minh. Nếu G là partial 1-tree thì mỗi thành phần liên thông của nó cũng vậy. Theo bổ đề cạnh, mỗi thành phần liên thông là cây, nên G là rừng.

Ngược lại, nếu $G = (V, E)$ là rừng, ta có thể xây phân rã cây độ rộng 1 như sau. Tập nút I có một nút cho mỗi đỉnh của G và một nút cho mỗi cạnh của G . Với cạnh (u, v) , nối nút đại diện cạnh này với hai nút đại diện u và v , rồi nối thêm tùy ý để T liên thông. Bag của nút đại diện đỉnh u là $\{u\}$, còn bag của nút đại diện cạnh (u, v) là $\{u, v\}$.

5.4.2 Thu gọn partial 2-tree

Xét ba thao tác trên đồ thị G :

- gộp cạnh bội;
- xóa đỉnh bậc 0 hoặc bậc 1;
- thu gọn đỉnh bậc 2: nối hai láng giềng của nó bằng một cạnh rồi xóa nó.

Mỗi lần thực hiện một thao tác gọi là một lần thu gọn.

Bổ đề 4.2.1. Nếu G' thu được từ G bằng một lần thu gọn, thì G là partial 2-tree khi và chỉ khi G' là partial 2-tree.

Hai thao tác đầu hiển nhiên. Với thao tác thứ ba, giả sử xóa u và hai láng giềng là v, w . Nếu G có phân rã cây độ rộng 2, xóa u khỏi các bag chứa nó và thêm v khi cần sẽ cho phân rã của G' . Ngược lại, từ phân rã của G' , lấy một bag chứa v, w , thêm một nút con mới có bag $\{u, v, w\}$, ta được phân rã cây độ rộng 2 của G .

Bổ đề 4.2.2. Nếu $G = (V, E)$ là partial 2-tree thì tồn tại $u \in V$ sao cho $d_G(u) \leq 2$. Chọn đỉnh có lev lớn nhất và dùng Bổ đề 3.0.1.

Một quá trình thu gọn cực đại là quá trình thực hiện liên tiếp các thao tác trên cho tới khi không còn thao tác nào làm được.

Định lý 4.2. Mọi quá trình thu gọn cực đại trên một partial 2-tree đều xóa sạch đồ thị.

Tính chất này gần trùng với khái niệm đồ thị nối tiếp-song song tổng quát; khác biệt là định nghĩa nối tiếp-song song tổng quát không cần xét thao tác xóa đỉnh bậc 0. Nói cách khác, đồ thị nối tiếp-song song tổng quát là partial 2-tree liên thông.

5.4.3 Cây nối tiếp-song song

Quá trình thu gọn trên có thể được mô tả bằng một cây ba nhánh có gốc. Mỗi lá đại diện cho một đỉnh hoặc cạnh ban đầu. Tại mọi thời điểm, mỗi đỉnh hoặc cạnh hiện có trong đồ thị ứng với một nút hiện tại trên cây này. Mỗi lần thu gọn tạo một nút mới x :

- gộp cạnh bội: hai nút hiện tại của hai cạnh bội trở thành con của x , và cạnh mới hiện tại là x ;
- xóa đỉnh bậc 1: nếu xóa u kề v , thì các nút hiện tại của u , v và (u, v) trở thành con của x , còn v hiện tại là x ;
- thu gọn đỉnh bậc 2: nếu xóa u kề v, w , thì các nút hiện tại của u , (u, v) và (u, w) trở thành con của x , còn cạnh mới hiện tại là x .

5.4.4 Quy hoạch động dựa trên thu gọn

Nhiều bài khó trên đồ thị tổng quát có thể giải tốt trên partial 2-tree nhờ DP theo quá trình thu gọn. Ví dụ, xét bài SNOI2020 *Cây khung*: cho đồ thị vô hướng liên thông n đỉnh, m cạnh; biết rằng xóa một cạnh sẽ được cactus; cần đếm số cây khung modulo 998244353.

Có thể chứng minh đồ thị thỏa điều kiện là đồ thị nối tiếp-song song tổng quát. Với một cạnh hiện tại $e = (u, v)$, ghi f_e là số phương án trong phần đã thu gọn thành e sao cho u, v liên thông, và g_e là số phương án sao cho u, v không liên thông; các đỉnh bị xóa trong phần đó vẫn phải liên thông với ít nhất một trong u, v .

Với xóa đỉnh bậc 1, cạnh kề của đỉnh đó bắt buộc nằm trong cây khung, nên nhân đáp án với f_e . Với gộp hai cạnh bội e_1, e_2 thành e_0 , hai cạnh không thể đồng thời liên thông:

$$f_{e_0} = f_{e_1}g_{e_2} + f_{e_2}g_{e_1}, \quad g_{e_0} = g_{e_1}g_{e_2}.$$

Với thu gọn đỉnh bậc 2, hai cạnh kề không thể đồng thời không liên thông:

$$f_{e_0} = f_{e_1}f_{e_2}, \quad g_{e_0} = f_{e_1}g_{e_2} + f_{e_2}g_{e_1}.$$

Độ phức tạp là $O(n + m)$; bản chất thuật toán là DP từ dưới lên trên cây nối tiếp-song song.

5.4.5 Một số đồ thị khác

Đồ thị đầy đủ K_n có treewidth $n - 1$. Đồ thị phẳng có treewidth cỡ căn bậc hai. Cactus và đồ thị ngoài phẳng có treewidth không quá 2. Halin graph và Apollonian network có treewidth không quá 3. Với đồ thị chordal, treewidth bằng kích thước clique lớn nhất trừ 1 và có thể tính trong thời gian đa thức. Một định nghĩa tương đương khác của treewidth là: trong các đồ thị chordal chứa đồ thị gốc, lấy kích thước clique lớn nhất nhỏ nhất rồi trừ 1.

5.5 Treewidth bị chặn

5.5.1 Tìm phân rã cây độ rộng nhỏ

Tính treewidth của đồ thị tổng quát đã được chứng minh là NP-complete. Dù vậy, có nhiều thuật toán tham số theo treewidth:

- với đồ thị n đỉnh có treewidth k , có thuật toán tìm phân rã cây độ rộng k trong thời gian $n^{k+O(1)}$;
- có thuật toán thời gian $2^{\tilde{O}(k^3)} n$;
- có thuật toán cho phân rã độ rộng không quá $5k + 4$ trong thời gian $2^{O(k)} n$;
- có thuật toán xấp xỉ độ rộng $O(k\sqrt{\log k})$ trong thời gian đa thức.

Điểm đáng chú ý là nhiều thuật toán có độ phức tạp tăng rất nhanh theo k , nhưng tuyến tính hoặc gần tuyến tính theo n khi k được xem là hằng số.

5.5.2 Ý nghĩa của treewidth bị chặn

Với mọi hằng số k , việc kiểm tra một đồ thị có phân rã cây độ rộng k hay không, và dựng nó nếu có, có thể làm tuyến tính theo số đỉnh. Ngoài ra, khi giới hạn đồ thị đầu vào có treewidth không vượt quá một hằng số, nhiều bài toán NP có thể giải trong thời gian đa thức. Với một lớp lớn các bài dễ trên cây, ta có thể làm DP trên phân rã cây để đạt độ phức tạp tốt.

Định lý 5.5 (Courcelle). Nếu một bài toán đồ thị biểu diễn được bằng logic bậc hai đơn nguyên, thì trên đồ thị treewidth bị chặn, bài toán đó giải được trong thời gian tuyến tính.

Ví dụ gồm tô màu đồ thị, chu trình Hamilton, clique lớn nhất và tập độc lập lớn nhất. Phần sau minh họa bằng bài tập độc lập lớn nhất.

5.5.3 Phân rã cây hoàn hảo

DP trên phân rã cây tổng quát thường khá rườm rà. Nếu phân rã cây có thêm cấu trúc, DP sẽ dễ viết hơn.

Định nghĩa 5.1 (phân rã cây hoàn hảo). Một phân rã cây (X, T) của $G = (V, E)$ gọi là hoàn hảo nếu T là cây nhị phân có gốc và mỗi nút $i \in I$ thuộc đúng một trong bốn loại sau:

- lá: không có con và $|X_i| = 1$;
- nút đưa vào: có một con j và $X_i = X_j \cup \{v\}$;
- nút quên: có một con j và $X_i = X_j \cup \{v\}$;
- nút hợp: có hai con j, k và $X_i = X_j = X_k$.

Định lý 5.6. Cho một phân rã cây độ rộng bị chặn của G , có thuật toán tuyến tính biến nó thành một phân rã cây hoàn hảo cùng độ rộng.

Ý tưởng chuyển đổi như sau. Nếu một nút lá có bag chứa hơn một đỉnh, thêm nút con và mỗi lần bỏ một đỉnh khỏi bag. Nếu một nút có một con nhưng hai bag khác nhau quá nhiều, chèn các nút trung gian để mỗi bước chỉ đưa vào hoặc quên một đỉnh. Nếu một nút có hai con nhưng bag của con chưa bằng bag của nó, chèn nút trung gian có bag bằng bag của cha. Nếu một nút có nhiều hơn hai con, tách dần thành cây nhị phân bằng các nút trung gian.

5.5.4 Tập độc lập lớn nhất

Cho đồ thị vô hướng $G = (V, E)$, một tập $I \subseteq V$ gọi là độc lập nếu mọi $u, v \in I$ đều không có cạnh nối. Biết một phân rã cây hoàn hảo độ rộng w là hằng số, ta giải tập độc lập lớn nhất bằng DP từ dưới lên.

Với mỗi nút i và mỗi $S \subseteq X_i$, đặt $\text{dp}_{i,S}$ là kích thước tập độc lập lớn nhất trong phần cây con của i , với điều kiện các đỉnh được chọn trong bag X_i đúng là S . Nếu S không độc lập thì trạng thái vô hiệu.

Các chuyển tiếp:

- nút lá: $\text{dp}_{i,\emptyset} = 0$ và $\text{dp}_{i,X_i} = 1$;
- nút đưa vào, với đỉnh đưa vào là v và con j :

$$\text{dp}_{i,S} = \text{dp}_{j,S \setminus \{v\}} + [v \in S];$$

- nút quên, với đỉnh bị quên là v và con j :

$$\text{dp}_{i,S} = \max(\text{dp}_{j,S}, \text{dp}_{j,S \cup \{v\}});$$

- nút hợp, với hai con j, k :

$$\text{dp}_{i,S} = \text{dp}_{j,S} + \text{dp}_{k,S} - |S|.$$

Độ phức tạp là $O(2^w w^2 n)$.

5.5.5 Đường đi ngắn nhất

Xét đồ thị vô hướng không trọng số và nhiều truy vấn đường đi ngắn nhất giữa s, t . Với phân rã cây (X, T) , gọi m là LCA của top_s và top_t trong T .

Định lý 5.7. Nếu xóa mọi đỉnh trong X_m khỏi G , thì s, t không còn liên thông.

Do đó mọi đường đi ngắn nhất từ s tới t phải đi qua ít nhất một đỉnh trong X_m , và

$$\text{dist}(s, t) = \min_{i \in X_m} \text{dist}(s, i) + \text{dist}(i, t).$$

Nếu cây phân rã có chiều cao h và độ rộng w , ta có thể BFS để tiền xử lý khoảng cách cho các cặp có quan hệ tổ tiên trong $O(hn)$, rồi trả lời mỗi truy vấn trong $O(w)$.

5.6 Thuật toán phân rã cây của Bodlaender

Phần này phác thảo thuật toán tuyến tính của Bodlaender năm 1996 cho đồ thị có treewidth bị chặn. Thuật toán đầy đủ khá dài; bài viết chỉ nêu các ý tưởng và sản phẩm phụ quan trọng.

Bổ đề 6.1.1. Với hai hằng số bị chặn k, l , nếu đã có phân rã cây độ rộng l của G , thì có thể kiểm tra $\text{tw}(G) \leq k$ hay không và nếu có thì dựng phân rã cây độ rộng không quá k , trong thời gian tuyến tính.

Dựa trên bổ đề này, cần chứng minh rằng với hằng số k , ta có thể hoặc kết luận $\text{tw}(G) > k$, hoặc tuyến tính thu nhỏ đồ thị theo một tỉ lệ cố định rồi đệ quy. Thuật toán xét hai trường hợp:

1. Nếu có đủ nhiều “đỉnh xanh”, cực đại matching của G đủ lớn; thu gọn các cạnh trong matching rồi đệ quy.
2. Nếu không, sau khi thêm một số cạnh không làm đổi treewidth, sẽ có đủ nhiều “đỉnh I-simplex”; xóa các đỉnh này rồi đệ quy.

5.6.1 Chuẩn bị

Bổ đề 6.2.1. Với một phân rã cây (X, T) của $G = (V, E)$:

1. nếu $W \subseteq V$ tạo thành clique trong G , thì tồn tại $i \in I$ sao cho $W \subseteq X_i$;
2. nếu $W_1, W_2 \subseteq V$ tạo thành đồ thị hai phía đầy đủ, thì tồn tại $i \in I$ sao cho $W_1 \subseteq X_i$ hoặc $W_2 \subseteq X_i$.

Bổ đề 6.2.2. Nếu tồn tại bag chứa cả u và v , thì (X, T) cũng là phân rã cây của đồ thị $G + (u, v)$.

Bổ đề 6.2.3. Nếu $|N_G(u) \cap N_G(v)| > k$, thì

$$\text{tw}(G) \leq k \iff \text{tw}(G + (u, v)) \leq k.$$

Lý do là nếu hai đỉnh có quá nhiều láng giềng chung, trong mọi phân rã cây độ rộng k hoặc phải có bag chứa u, v , hoặc tạo ra clique quá lớn.

Định nghĩa 6.2.1 (phân rã cây đều). Một phân rã cây (X, T) của $G = (V, E)$ gọi là đều nếu với mọi $i \in I$, $|X_i| = k + 1$, và với mọi cạnh cây $(i, j) \in F$, $|X_i \cap X_j| = k$.

Từ một phân rã cây độ rộng bị chặn, có thể biến đổi tuyến tính thành phân rã cây đều cùng độ rộng.

5.6.2 Đỉnh xanh

Chọn hai hằng số $0 < c_1, c_2 < 1$. Đặt

$$d = \max \left(k^2 + 4k + 4, \left\lceil \frac{2k}{c_1} \right\rceil \right).$$

Đỉnh có bậc không quá d gọi là đỉnh bậc thấp; đỉnh có bậc lớn hơn d gọi là đỉnh bậc cao. Một đỉnh bậc thấp kề với ít nhất một đỉnh bậc thấp khác gọi là đỉnh xanh.

Bổ đề 6.3.1. Số đỉnh bậc cao nhỏ hơn $c_1|V|$. Điều này suy từ cận số cạnh của partial k -tree.

Bổ đề 6.3.2. Nếu G có n_f đỉnh xanh, thì mọi cực đại matching của G chứa ít nhất $n_f/(2d)$ cạnh.

Chứng minh. Với một cực đại matching M , mỗi đỉnh xanh hoặc đã là đầu mút của cạnh trong M , hoặc kề với một đỉnh xanh nằm trên M ; nếu không, matching chưa cực đại. Gán đóng góp của đỉnh xanh chưa ghép cho một đỉnh xanh đã ghép kề nó. Mỗi đỉnh như vậy nhận nhiều nhất d đóng góp, nên M có ít nhất n_f/d đỉnh xanh và ít nhất $n_f/(2d)$ cạnh.

Thu gọn mọi cạnh trong một matching M được G' . Thu gọn cạnh không tăng treewidth; ngược lại, từ phân rã cây độ rộng k của G' có thể phục hồi phân rã cây độ rộng $2k + 1$ của G bằng cách thay mỗi đỉnh thu gọn bằng hai đầu mút cũ. Sau đó dùng Bổ đề 6.1.1 để giảm độ rộng về k . Vì vậy khi có đủ nhiều đỉnh xanh, ta thu gọn matching và đệ quy.

5.6.3 Đỉnh I-simplex

Định nghĩa 6.4.1 (đồ thị bổ sung). Với mọi cặp u, v có ít nhất $k + 1$ láng giềng chung bậc thấp, thêm cạnh (u, v) . Đồ thị thu được gọi là đồ thị bổ sung G' . Theo Bổ đề 6.2.3, $\text{tw}(G) \leq k$ khi và chỉ khi $\text{tw}(G') \leq k$.

Định nghĩa 6.4.2. Một đỉnh v gọi là I-simplex nếu trong đồ thị bổ sung, các láng giềng của nó tạo thành clique, và trong đồ thị gốc nó là đỉnh bậc thấp nhưng không phải đỉnh xanh.

Định nghĩa 6.4.3. Với một phân rã cây (X, T) của G , một đỉnh bậc thấp không xanh v gọi là đỉnh đỏ nếu tồn tại $i \in I$ sao cho $N_G(v) \subseteq X_i$. Đỉnh đỏ chỉ dùng trong phân tích, thuật toán không cần tính chúng trực tiếp.

Bài viết chứng minh một chuỗi bổ đề: trong phân rã cây đều, mỗi lá của T và mỗi đường dài đủ lớn gồm các nút bậc 2 đều bảo đảm tồn tại một đỉnh xanh hoặc đỏ xuất hiện cục bộ; mọi cây có nhiều lá hoặc nhiều đường như vậy; số tập quan trọng tạo bởi các đỉnh bậc cao không vượt quá số đỉnh bậc cao. Từ đó suy ra nếu đỉnh xanh không đủ nhiều, thì phải có đủ nhiều đỉnh I-simplex.

Cụ thể, nếu $G = (V, E)$ có $\text{tw}(G) \leq k$, có n_f đỉnh xanh và n_h đỉnh bậc cao, thì số đỉnh I-simplex ít nhất là

$$\frac{|V|}{2k^2 + 6k + 8} - 1 - n_f - \frac{1}{2}k^2(k + 1)n_h.$$

Bổ đề 6.4.5. Giả sử xóa mọi đỉnh I-simplex khỏi đồ thị bổ sung được G' , và (X, T) là một phân rã cây của G' . Với mỗi đỉnh I-simplex bị xóa v , tồn tại bag X_i sao cho $N_G(v) \subseteq X_i$.

Do đó từ phân rã cây của đồ thị đã xóa các đỉnh I-simplex, ta phục hồi nhanh phân rã cây cùng độ rộng của đồ thị ban đầu bằng cách thêm cho mỗi v một bag $\{v\} \cup N_G(v)$ gắn vào bag chứa $N_G(v)$.

5.6.4 Luồng thuật toán

Bài toán cần giải là: cho đồ thị vô hướng $G = (V, E)$ và hằng số k , hãy dựng phân rã cây độ rộng không quá k , hoặc báo rằng $\text{tw}(G) > k$. Nếu $|V|$ nhỏ hơn một hằng số đủ lớn, dùng tìm kiếm trực tiếp.

Trước hết kiểm tra

$$|E| \leq k|V| - \frac{1}{2}k(k + 1).$$

Nếu không thỏa, chắc chắn vô nghiệm.

Nếu số đỉnh xanh ít nhất $|V|/(4k^2 + 12k + 16)$:

- tìm một cực đại matching M ;
- thu gọn các cạnh trong M để được G' ;
- đệ quy trên G' , nếu vô nghiệm thì G cũng vô nghiệm;
- phục hồi phân rã cây độ rộng $2k + 1$ của G ;
- dùng Bổ đề 6.1.1 để giảm về độ rộng k .

Ngược lại:

- dựng đồ thị bổ sung G' ;
- nếu tồn tại đỉnh I-simplex có bậc lớn hơn k , báo vô nghiệm;
- lấy tập S gồm mọi đỉnh I-simplex; nếu $|S| < c_2|V|$, báo vô nghiệm;
- đệ quy trên đồ thị xóa S ;
- với mỗi $v \in S$, thêm một bag $\{v\} \cup N_G(v)$ nối vào bag chứa $N_G(v)$.

Trong trường hợp thứ nhất, ta xóa được ít nhất một tỉ lệ cố định số đỉnh nhờ matching. Trong trường hợp thứ hai, ta xóa được ít nhất $c_2|V|$ đỉnh. Vì mỗi lần đệ quy đều thu nhỏ kích thước xuống còn nhiều nhất c_4n với $c_4 < 1$, độ phức tạp thỏa $T(n) = T(c_4n) + O(n)$, tức là tuyến tính.

5.7 Tổng kết

Bài viết xuất phát từ một số đồ thị đặc biệt, phân tích đặc trưng và tính chất của treewidth, giới thiệu ý nghĩa của treewidth bị chặn trong việc giải các bài toán kinh điển, rồi trình bày tư tưởng cốt lõi của một thuật toán phân rã cây mạnh. Nội dung chỉ mang tính bán phổ biến kiến thức; các liên hệ rộng hơn và những phương pháp phân rã mạnh hơn vẫn còn chờ độc giả tiếp tục tìm hiểu. Tác giả hy vọng treewidth sẽ được ứng dụng nhiều hơn trong thi tin học.

5.8 Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Wang Xingming của Trường Thập Nhất Bắc Kinh đã quan tâm và chỉ dẫn; cảm ơn gia đình đã ủng hộ; và cảm ơn các bạn đã trao đổi, thảo luận cùng tác giả.

5.9 Tài liệu tham khảo

- [1] Stefan Arnborg, Derek G. Corneil, and Andrzej Proskurowski. *Complexity of finding embeddings in a k -tree*. SIAM Journal on Algebraic and Discrete Methods, 8:277–284, 1987.
- [2] Hans L. Bodlaender. *A linear-time algorithm for finding tree-decompositions of small treewidth*. SIAM Journal on Computing, 25(6):1305–1317, 1996.
- [3] Hans L. Bodlaender, Pal Grønås Drange, Markus S. Dregi, Fedor V. Fomin, Daniel Lokshtanov, and Michał Pilipczuk. *An $O(c^k n)$ 5-approximation algorithm for treewidth*. FOCS, 2013.
- [4] Uriel Feige, Mohammad Taghi Hajiaghayi, and James R. Lee. *Improved approximation algorithms for minimum-weight vertex separators*. STOC, 2005.
- [5] Bruno Courcelle. *The monadic second-order logic of graphs I: recognizable sets of finite graphs*. Information and Computation, 85(1):12–75, 1990.
- [6] Hans L. Bodlaender and Ton Kloks. *Better algorithms for the pathwidth and treewidth of graphs*. ICALP, 1991.
- [7] Dian Ouyang, Lu Qin, Lijun Chang, Xuemin Lin, Ying Zhang, and Qing Zhu. *When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks*. SIGMOD, 2018.
- [8] Wikipedia. *Treewidth*.